

KairoX ERP — Production & Traceability

API Reference

Every endpoint of the production system, with purpose, roles, request, response and errors — enough to build the entire frontend against mocks, and enough for a backend developer to use as the contract of record.

Covers: Auth & Users · Employees · Clients · Imports · Barcode · Materials · **Cutting** · Production · **Store** (*replaces Drawers*) · **Inspections** · **Job Work** · Attendance · Wages · Analytics · Dashboard

Updated 2026-09-21, for the paging release and, before it, the Cutting V2 / Store / Reject-Rework / Job-Work release. **Start at §0** — it lists everything new, everything that changed shape, and the things that will break a screen if you do nothing. The two current ones: **twenty list endpoints now return 50 rows instead of all of them**, and **four routes were removed** (§0.0).

Checked against the running app. The 154 `/api/v1` endpoints indexed in §6 are the routes the app actually serves, verified against `/openapi.json`; `postman/` is generated from that same schema, so a path in one and not the other is a bug in this document.

Companion document: *KairoX Production-Event System Guide* — the business logic, module architecture and screen design behind these endpoints.

How to use this document

Frontend developers. Every endpoint carries a complete, realistic mock response. Copy them into a mock server and build the whole UI without a backend. §40 is a fixture pack with consistent IDs, so one mocked order flows end to end. §41 gives the exact call sequences for every user journey.

Backend developers. This is the contract of record: exact paths, exact role gates, exact status codes, exact response keys.

Everyone. Endpoints are grouped by module, in the order an order travels through the factory.

0. WHAT CHANGED — read this first

This section is written for the **frontend developer**. It lists everything that is new, everything that changed shape, and the things that will break a screen if you do nothing. Everything else in this document is still accurate.

Newest release first. §0.0 is the current one; §0.1 onward is the Cutting V2 / Store / Reject-Rework / Job-Work release before it, still accurate and still worth reading if you have not migrated off drawers.

0.0 BREAKING — the paging release (2026-09-19)

1. Twenty list endpoints now return 50 rows, not all of them

A screen that assumed an unbounded array is now silently showing the first page. Nothing errors. Nothing warns. The list is just short.

`GET /production/events` with no filter used to mean "serialise the whole production history", and `production_event` gains a row per piece per stage and never shrinks. That is survivable after a few weeks of running and fatal after a year — a request that times out while holding a pooled database connection, which is this service's real concurrency ceiling.

What to do: read `has_more` and page. `limit` defaults to `50`, caps at `200`, and `?limit=999999` is a `422` rather than a way back to the old behaviour. **\$3.1 lists all twenty endpoints, the Page envelope, the three that page with an older shape, and the counts that are deliberately not page-scoped.**

The trap worth knowing before you build the pager: on `GET /production/skus/{id}/pieces` the `total / done / pending / closed` counts are **SKU-wide**, not page-wide — `closed` withdraws a style from the scan screen, and deriving it from one page would tell a manager on page 1 that a 900-piece SKU was finished. `blocked` is the exception, and it is tagged `blocked_scope` so you can label it honestly. Do not sum header counts across pages.

2. Four routes were removed

They are unrouted and **404**. Each is struck out in the endpoint index (§6) and its own section says why:

Gone — 404	What replaces it
~~ POST <code>/users/clients</code> ~~ (§8.3)	Nothing. It was the only way to mint a <code>CLIENT</code> login, and Phase 1 gives a buyer nothing to do inside the app. <code>POST /clients</code> creates the client record and is unaffected.
~~ GET <code>/attendance/me/status</code> ~ ~ (§18.8)	Nothing. The shift countdown suited the shop floor, and the shop floor has no login. <code>GET /attendance/me</code> still returns a staff login's own history.
~~ GET <code>/attendance/config</code> ~~ (§18.9)	Nothing. Shift policy is configured once, server-side.
~~ PATCH <code>/attendance/config</code> ~~ (§18.9)	Nothing. It went with the read: an edit form that cannot load its current values overwrites policy with whatever the frontend held.

The shift policy itself is unaffected — `is_late` / `is_short` / `is_overtime` are still set on every punch and the wage run still prices against it. What is gone is the API that read and wrote the row.

0.1 The one-paragraph summary

Drawers are gone. There were 200 physical boxes; a style released 100+ garments, the pool ran dry, and the DM had to re-allocate by hand. The store is now a *state on the garment*, so it has no capacity and nothing runs out. The store scan dropped from three scans to two.

Cutting got a spreadsheet. Leather is now tracked hide by hide. The cutting manager opens a grid, the system allocates ~10 sheets per garment, he edits it, approves it — and the cutter's scan pulls all of it in without typing anything.

Three new things: rejecting a piece and sending it back for rework; sending garments to an outside factory and getting them back; and accessories that only reach the garment size they are for.

0.2 BREAKING — two things to fix before you ship

1. `/api/v1/drawers/*` is gone (404)

Every drawer route is withdrawn. Replace as follows:

Old	New
POST /drawers/store-scan	POST /store/scan — no drawer_id , and the drawer scan step disappears from the UI
POST /drawers/send	POST /store/send — takes piece_ids , not drawer_ids
GET /drawers	GET /store/pieces
GET /drawers/by-code/{code}	GET /store/pieces/{piece_code}
GET /drawers/{id}	GET /store/pieces/{piece_code}
POST /drawers/{id}/receive	nothing — the scan is the receipt now
GET/POST /drawers/pool , /drawers/allocate-waiting	nothing — there is no pool

Old DRAWER barcodes still **resolve** (so a printed sticker does not 404), but they return `next_expected_scan: null` and their state is frozen history.

2. drawer in production responses is now always null

GET /production/piece-state and the piece-checklist rows still return a `drawer` key, but it is `null` for one release so nothing crashes. The real data moved to a new `store` block:

```
{
  "store": {
    "state": "holding_both",          //
waiting|merged|holding_leather|holding_lining|holding_both|received|sended
    "holding": "HOLDING BOTH",        // ready-to-render caption
    "leather_in": true,
    "lining_in": true,
    "accessories_in": false
  },
  "drawer": null,                    // REMOVED NEXT RELEASE - migrate off it
  "drawer_code": null
}
```

Do this now: search your codebase for `.drawer` and `drawer_code` . Anything reading `drawer.code` will render blank rather than crash, which is worse — it looks like missing data instead of a bug.

0.3 POST /production/log — three new rejection buckets

The request is **unchanged**. The response gained three per-piece buckets alongside `sequence_blocked` / `skill_blocked` / `merge_blocked` :

Bucket	Means	What the operator should be told
assignment_blocked	the scanned worker is not the cutter this row was approved for	"This piece was approved for a different cutter."
rejected_blocked	a rejection on this garment is waiting for the DM	"Rejected at PASTING — waiting for the DM."
offsite_blocked	the garment is physically at an outside factory	"Out at ABC Tailors since 12/09."

Every one of them is **per piece**, like the existing gates: one bad garment never loses the tray it was scanned with.
The `blocked[]` array is what you should actually render. It carries the reason sentence per piece and has done since before this release:

```
"blocked": [
  {"piece": "PC-2234", "gate": "offsite", "stage": "LINE_STITCHING",
   "reason": "PC-2234 is out at ABC Tailors for LINE_STITCHING since 2026-09-12. Book it back in
before logging anything on it here."}
]
```

Design note. Do not build a separate UI branch per bucket. Render `blocked[]` as a list of rows — piece code, a badge from `gate`, and `reason` verbatim. The reasons are written to be shown to the floor as-is; they name the fix, not just the problem. New gates then appear in your UI with no work.

0.4 NEW: Cutting grid — `/cutting/*`

Roles: CUTTING_MANAGER, DM, MD.

This replaces Kumar's Excel. One row = **one garment**.

GET `/cutting/grid?style_id={uuid}&colour={string}`

```
{
  "style_id": "...", "style_name": "CLERMONT", "colour": "NAVY",
  "colours": ["NAVY", "WHISKY"],           // fills the colour picker
  "uncut_pieces": 12,                      // garments with no row yet
  "present_cutters": [                    // ONLY people checked in today
    {"employee_id": "...", "name": "MAJID", "designation": "CUTTER"}
  ],
  "warnings": [],
  "rows": [
    {
      "row_id": "...", "piece_id": "...", "piece_code": "PC-2234",
      "article": "SUEDE-A32", "colour": "NAVY", "size": "L", "rc_no": "1072",
      "cutter_employee_id": "...", "cutter_name": "MAJID",
      "work_date": "2026-09-18", "status": "DRAFT",
      "target_dcm": 430.0, "target_source": "size_baseline",
      "total_dcm": 438.0, "sheet_count": 10,
      "sheets": [
        {"sheet_id": "...", "code": "LS-000001", "dcm": 43.0, "status": "ALLOCATED"}
      ],
      "approved_at": null, "logged_at": null,
      "warnings": []
    }
  ]
}
```

target_source matters for the UI. `"style_spec"` means a human measured this style — show the number plainly. `"size_baseline"` means the system estimated it from the garment size — show it greyed or with a hint, because the manager is *expected* to correct it.

present_cutters is the whole cutter dropdown. Do not offer anyone else: production refuses to log an absent worker, so a row assigned to someone who did not clock in collects the leather and then fails at the scan.

POST /cutting/rows/generate

```
// request
{"style_id": "...", "colour": "NAVY", "cutter_employee_id": "...",
 "material_lot_id": null,    // omit → resolved from the style's article+colour
 "limit": null, "work_date": null, "allocate": true}

// response
{"created": 12, "rows": [ ...RowRead... ],
 "warnings": ["PC-2240: only 114 dcm of hide was left against a target of 430 – the stock ran out
 partway through. Add sheets before approving."]}
```

Safe to press twice — rows are only created for pieces that have none.

409 when several leather lots match the style's article+colour. Send `material_lot_id` to disambiguate; the system will not guess which hides to spend.

Editing a row

Call	Body	Use
PATCH /cutting/rows/{id}	{cutter_employee_id?, size?, rc_no?, article?, colour?, work_date?, note?}	edit any cell
POST /cutting/rows/{id}/sheets	{sheet_code} or {sheet_id} or {dcm, material_lot_id}	the cutter needed one more
DELETE /cutting/rows/{id}/sheets/{sheet_id}	—	he handed one back
PATCH /cutting/rows/{id}/sheets/{sheet_id}	{dcm}	correct a measurement

All four return the whole updated `RowRead`, so **re-render the row from the response** rather than patching local state.

POST `.../sheets` has three doors on purpose: `sheet_code` when he scans the hide, `sheet_id` when he picks it from a list, `dcm` when the delivery was never sheeted and the hide has to be created now.

POST /cutting/rows/{id}/approve

```
{"row": { ...RowRead, "status": "APPROVED"... },
 "message": "Approved: 10 sheet(s), 438 dcm. Scan the piece on the cutting screen to log it."}
```

Approving freezes the row and moves its hides to `ISSUED`. After that every edit returns **409** — use `POST /cutting/rows/{id}/reopen?reason=...` (audited).

Re-approving is a **no-op, not an error** — a manager who is unsure whether the first tap landed will press it again, and they should not be punished for it.

409 before approve if: no sheets on the row, or no cutter assigned.

Screen design. Build this as a real spreadsheet, not a form-per-row: one table, sticky style/colour header, sheet DCMs as editable cells across the row, a derived total column you never let anyone type into, and one APPROVE button per row. The factory has used a spreadsheet for years — matching the shape they know is worth more than a prettier layout.

0.5 NEW: The store — /store/*

Scan roles: STORE_MANAGER, STITCHING_MANAGER, DM, MD. **Read roles:** all floor roles + HR.

POST /store/scan — two scans, not three

```
// request - the worker and the garment. No drawer.
{"employee_barcode": "EMP-4F2A11", "piece_barcode": "PC-2234",
 "part": null,           // null → inferred. "ACCESSORY" must always be explicit.
 "lines": null}         // optional: a partial/substituted kit issue

// response
{
  "piece_code": "PC-2234",
  "store_state": "holding_both",
  "holding": "HOLDING BOTH",
  "leather_in": true, "lining_in": true, "accessories_in": false,
  "part_stored": "LINING", "part_inferred": true,
  "complete": false,
  "awaiting": ["ACCESSORIES"],           // what is still owed - render as chips
  "auto_received": false,
  "ready_for_received": false,
  "sent": false,
  "needs_lining": true,
  "lining_reason": "its style name contains 'KNIT'",
  "kit": { ...the accessory checklist, on EVERY scan... },
  "next_action": "Waiting for its accessories - scan the kit to issue them.",
  "warnings": []
}
```

part is optional and you should usually omit it. The system infers LEATHER vs LINING from the garment's own history — the operator should not be asked a question the system can answer. **ACCESSORY** is the exception and must always be sent explicitly, because an inferred accessory scan *spends stock*.

next_action is a finished sentence. Render it directly as the primary line on the scan result. **awaiting** is the same information as chips if you want both.

kit now rides every scan, not just accessory scans — the operator holding the lining is the person best placed to fetch the buttons too, and they will not open a second screen to find out they are owed.

POST /store/send

```
// request
{"piece_ids": ["..."], "piece_barcodes": ["PC-2234"]} // either or both

// response
{"count_sent": 2, "sent": ["PC-2234", "PC-2235"],
 "not_ready": [{ "piece": "PC-2236", "missing": "lining", "state": "holding_leather",
                  "reason": "PC-2236 is still awaiting its lining. It takes a lining because its style
name contains 'KNIT'." }],
 "not_found": [],
 "message": "2 garment(s) sent to line-stitching; 1 not ready"}
```

Partial accept. One incomplete garment never loses the complete ones selected with it. Render **sent** and **not_ready** as two lists — do not treat a non-empty **not_ready** as a failed request.

`GET /store/pieces?state=&style_id=&limit=` • `GET /store/pieces/{piece_code}`

The lookup is deliberately open to **every floor role and HR**. The DM assigns somebody to place garments who has no DM login; under the old routes they could not look a barcode up at all and had to go and find the DM.

Screen design. The store screen is now a *list of garments*, not a grid of boxes. Group by `store_state`, and lead each row with `holding` and `next_action`. The old drawer grid (200 numbered cells) has nothing to render any more — delete it rather than porting it.

0.6 NEW: Reject & rework — `/inspections/`*

Raise: every manager + HR. **Approve/decline:** DM, MD only.

A garment finished FUSING and was rejected at PASTING. Before this release there was no way to send it back; the only route was a database edit.

`POST /inspections`

```
// a PASS – closed immediately, nobody approves it
{"piece_barcode": "PC-2234", "found_at_stage": "PASTING", "verdict": "PASS"}

// a REJECT – goes to the DM
{"piece_barcode": "PC-2234",
 "found_at_stage": "PASTING",
 "verdict": "REJECT",
 "action": "REDO",                // FIX = repair here | REDO = send it back
 "return_to_stage": "FUSING",     // required for REDO; the rejector chooses
 "defect_type": "WORKMANSHIP",    // WORKMANSHIP | PRODUCT_DAMAGE
 "responsible_employee_id": "...", // REQUIRED for WORKMANSHIP, REFUSED for PRODUCT_DAMAGE
 "responsible_stage": "FUSING",
 "reason": "fusing lifted at the seam"}
```

The accountability rule is enforced, so build the form around it. Make `defect_type` the first choice, and let it drive the rest:

- **WORKMANSHIP** → reveal a *required* employee + stage picker. A 422 comes back if they are missing.
- **PRODUCT_DAMAGE** → *hide* the employee picker entirely. Sending one is a 422: a flawed hide is the supplier's problem, and putting it against a worker who did nothing wrong teaches the floor to stop reporting damage at all.

`return_to_stage` should be a dropdown of stages the piece has already passed — the API 409s on anything else, and you can populate it from `GET /production/piece-state`'s `completed_stages`.

`POST /inspections/{id}/approve` • `/decline`

Body `{"note": "..."}` is optional. Only after **approve** does the garment move.

What happens after approval

- The piece re-walks **everything the redo invalidated**: redo FUSING on a piece rejected at LINE_STITCHING and it must be fused, pasted and line-stitched again, in order.
- Those events come back with `is_rework: true`, so cost is separable.
- While the rejection is **pending**, the piece is `rejected_blocked` in the production log — it stops moving until the DM decides.

GET /inspections?status=&limit= — the DM's queue

GET /inspections/pieces/{piece_code} — one garment's history

GET /inspections/responsibility?employee_id=

```
[{"employee_id": "...", "employee": "FATIMA", "stage": "FUSING", "rejections": 2}]
```

WORKMANSHIP only. Product damage never appears here.

0.7 NEW: Job work (outsourcing) — /jobwork/*

Dispatch/receive: DM, MD. **Read:** all floor roles + HR.

POST /jobwork/vendors · **GET /jobwork/vendors?active_only=true**

```
{"name": "ABC Tailors", "contact": "98000 00000", "note": null}
→ {"vendor_id": "...", "name": "ABC Tailors", "contact": "...", "note": null, "is_active": true}
```

Registering the same name twice returns the original, not a 409.

POST /jobwork/dispatch

```
// request
{"vendor_id": "...", "stage": "LINE_STITCHING",
 "piece_ids": [...],
 "expected_back": "2026-09-25",
 "rate_per_piece": 45.0, "currency": "INR",    // OPTIONAL - omit if not per-piece
 "note": null}

// response
{"job_id": "...", "vendor": "ABC Tailors", "stage": "LINE_STITCHING",
 "status": "OUT", "dispatched_at": "...", "expected_back": "2026-09-25",
 "rate_per_piece": 45.0, "currency": "INR",
 "pieces_out": 2, "pieces_back": 0, "pieces_rejected": 0, "pieces_short": 0,
 "cost": 0.0,                      // rate × pieces that CAME BACK
 "overdue": false,
 "skipped": [{"piece_id": "...", "piece": "PC-9", "reason": "already out at XYZ Stitching"}]}
```

POST /jobwork/{job_id}/receive

```
// request - omit piece_ids and everything still out is treated as returned
{"piece_ids": null,
 "rejected_ids": [...],    // came back badly done
 "short_ids": [],         // never came back
 "work_date": null}
```

Returns the same job shape with `status` now `PARTIAL` or `RETURNED`.

cost counts only pieces that came back. Rejected and short are never paid for, and they are kept apart on purpose: a missing garment is a loss to chase with the vendor, a bad one is a quality conversation.

GET /jobwork?status=&vendor_id=&overdue=true&limit=

Screen design. Two views earn their place: *what is out right now* (group by vendor, show `pieces_out` and days elapsed) and *what is late* (`?overdue=true`). Put the overdue count somewhere permanent — a dispatch nobody chases is how garments go missing, and that is the whole reason this module exists.

0.8 CHANGED: Materials — /materials/*

Leather is now tracked hide by hide

POST /materials/lots and POST /materials/receive accept an optional `sheets` array. **LEATHER only** — a 422 comes back for anything else, because a 5,000-button packet is one barcode and a count, not 5,000 labels.

```
// request addition
{"category": "LEATHER", "article": "SUEDE-A32", "colour": "NAVY",
 "attributes": {"thickness": "1.2", "dcm": 552},
 "sheets": [{"dcm": 43}, {"dcm": 47}, {"dcm": 47}]}

// response addition – the labels to print
"sheets": [{"sheet_id": "...", "code": "LS-000001", "dcm": 43.0,
            "status": "IN_STOCK", "cutting_row_id": null}]
```

POST /materials/receive also returns `sheet_reconciliation` when the delivery was sheeted:

```
{"sheets_total": 8, "sheets_by_status": {"IN_STOCK": {"count": 8, "dcm": 363.0}},
 "sheet_dcm_in_store": 363.0, "lot_on_hand": 363.0,
 "difference": 0.0, "reconciled": true}
```

A mismatch **warns, it does not block** — refusing a delivery over two decimetres of measurement slop is how a store learns to stop sheeting.

GET /materials/lots/{id} gained five fields

```
"received": 390.0,           // everything that EVER arrived ← was missing entirely
"rejected": 15.0,           // supplier quality history
"deliveries": 3,
"sheets_total": 12,
"sheets_by_status": {"IN_STOCK": {"count": 10, "dcm": 460.0},
                     "CONSUMED": {"count": 2, "dcm": 92.0}}
```

This fixes the "Received shows 0" bug. It showed 0 because the field did not exist — nothing ever summed the receipts. `received - on_hand = consumed` now reconciles exactly, so you can show all three on the stock card.

New reads

Endpoint	Returns
GET /materials/lots/{id}/history	every delivery of that lot, newest first
GET /materials/leather-by-style?style_id=	arrived / consumed / available per style, split original vs rework
GET /materials/pieces/{piece_id}/consumption	what one garment took, hide by hide

```
// leather-by-style row
{"style_name": "CLERMONT", "article": "SUEDE-A32", "colour": "NAVY", "uom": "dcm",
 "arrived": 1000.0, "consumed": 920.0,
 "consumed_original": 860.0, "consumed_rework": 60.0,    // ← the honest split
 "on_hand": 80.0, "reserved": 0.0, "available": 80.0,
 "pieces": 2, "per_piece": 460.0}
```

HR can now write

HR was read-only on stock, which is why a wrong lot got fixed in the database by hand. HR can now create, edit, adjust, delete and receive. HR still **cannot** approve a PO-mismatch substitution (a costing decision) or run payroll.

Accessories only reach their own size

`style_material_spec` lines gained `garment_size`. A line with `garment_size: "L"` reaches only size-L garments; `null` means every size.

When the DM adds an accessory whose material `size` reads as a garment size (`S / M / L / XL / XXL` or `38 – 62`), `garment_size` defaults to it automatically — so a zip labelled **L** matches L garments with no extra data entry. A **60cm** zip or an **18L** button is correctly *not* read as a garment size and stays on every size.

Design note. On the recipe grid, show `garment_size` as its own column separate from `size`. They look alike and mean different things: `size` is the material's size (60cm zip), `garment_size` is which jackets it belongs on.

0.9 CHANGED: Barcode — `/barcode/resolve`

New type `LEATHER_SHEET`

```
{ "code": "LS-000003", "type": "LEATHER_SHEET", "active": true,
  "sheet": { "sheet_id": "...", "code": "LS-000003", "dcm": 47.0,
             "status": "IN_STOCK", "cutting_row_id": null },
  "lot": { ...the parent lot: article, colour, on_hand... },
  "next_expected_scan": "PIECE" }
```

Both blocks are returned: the scan must answer *which hide* **and** *what article and colour is it*, and only the lot knows the second half.

`next_expected_scan` no longer returns `"DRAWER"`

Scanned	Next
EMPLOYEE	"PIECE"
LEATHER_LOT / LINING_LOT / ACCESSORY_LOT / LEATHER_SHEET	"PIECE"
PIECE	null — the piece is the end of the scan, not the middle
DRAWER (legacy label)	null

If your scan flow has a "now scan the drawer" step, delete it.

0.10 CHANGED: Production — correcting a record

Two new endpoints replace editing the database by hand.

Call	Roles	Use
<code>PATCH /production/events/{id}/reassign?employee_id=&reason=</code>	managers + HR	wrong worker on real work — stock untouched
<code>DELETE /production/events/{id}?reason=</code>	DM, MD	the record should not exist — stock returned

`reason` is **required** on delete and the API 422s without it.

Both return **409** if the event falls inside a **CLOSED payroll run**: that run is the document the cash was counted against and is never recomputed, so the fix there is a payroll adjustment rather than a quiet edit underneath it.

Design note. Offer *Reassign* prominently and bury *Delete*. Reassign is the everyday correction; delete erases evidence and moves stock, and should feel like it.

0.11 CHANGED: Imports

`POST /imports/breakdown/{order_number}/release` no longer returns `pieces_waiting_for_drawer`, and `grow_drawer_pool` on the release body is ignored. There is no pool to wait for — the bottleneck is removed, so the "N pieces have no drawer" banner should come out of the upload screen.

0.12 CHANGED: Dashboards

`GET /dashboard/store/drawers/{id}` and `/movement` still exist but read frozen history. For live state use `GET /store/pieces`.

`GET /dashboard/direct-manager/pieces/{piece_code}` now accepts **either** code — the compact `PC-222223` a scanner returns, or the long `N1-BF27P010501-SUEDE_BOMBER-NAVY-S-034`. Previously only the long one worked, which is why the Analytics & Alert screen errored.

0.13 NEW: Correcting a person — employee reads and attendance CRUD

Two gaps closed. Both are about the same thing: **a record naming the wrong person is not a typo, it is a wage going to the wrong worker.**

`GET /employees/{employee_id}`

The single-employee read the edit screen opens on. Previously the only way to show one worker was to pull the whole roster and filter it in the browser.

Same gate as the roster: **every internal role; CLIENT and VIEWER are refused (403)** because the row carries phone and email. **HR / DM / MD additionally see `monthly_salary`** — everyone else gets the row without it.

`PATCH /attendance/{attendance_id}` — the re-allocation

This is the important one, and it is not "edit a field".

The real mistake on the floor is not "*nobody was here*". Security scans a card at the gate; the card said MAJID, the worker who actually did the day's cutting was SALIM. By the time anybody notices, MAJID has twelve cutting events against his name — and the wage for them, because **wages are computed from the events, not from the attendance.**

So changing `employee_id` **moves that day's production events with it, in the same action**, and the response says how many:

```
{
  "attendance_id": "a1b2c3d4-...",
  "employee_id": "e5f60718-...",
  "work_date": "2026-09-18",
  "check_in_at": "2026-09-18T03:32:10Z",
  "check_out_at": null,
  "is_late": false, "is_short": false, "is_overtime": false,
  "production_events_moved": 12,
  "message": "Re-allocated to the correct worker; 12 production event(s) for 2026-09-18 moved with it."
}
```

Field	Meaning
<code>employee_id</code>	re-allocation — the day's events move too
<code>check_in_at</code> / <code>check_out_at</code>	corrected times; <code>is_late</code> / <code>is_short</code> / <code>is_overtime</code> are recomputed , never left saying the old thing
<code>reason</code>	free text, written to the audit row

Roles: SECURITY · HR · MD · DM — every attendance operator. A gate operator who mis-scans must be able to fix it without going to find somebody.

Errors

Cod e	When
404	no such attendance record
409	the target worker is already marked present that day — one punch per worker per day is a database constraint, so this is said plainly instead of surfacing as a 500
409	the day falls inside a CLOSED payroll run

DELETE /attendance/{attendance_id}?reason=

Roles: HR · MD · DM **only**. SECURITY may *correct* a punch but not remove one — taking somebody off the day's roster decides whether they are paid for it.

`reason` is **required** and 422s when blank.

It refuses with 409 when the worker has production events that day, and the refusal names the better operation:

This worker has 3 production event(s) on 2026-09-18, so the work was really done — the usual cause is a swapped card, not an absent worker. Re-allocate the attendance to the right person instead (PATCH this record with the correct `employee_id`); the events move with it.

Deleting instead would leave those events dated to a day the worker was never marked present — which the production log's own presence gate would have refused to create.

Design note. Build the correction screen around **re-allocate**, with a worker picker, and put delete behind a second click. The count of events that will move belongs on the confirm button ("Move 12 events to SALIM"), because that number is the size of what the operator is about to do.

0.14 Suggested build order

If you are updating the frontend incrementally, this order keeps the app working at every step:

1. **Fix the breakages** (§0.2) — swap `/drawers/*` for `/store/*`, stop reading `drawer`. One afternoon, and nothing is broken afterwards.
2. **Render `blocked[]` generically** (§0.3) — one component, and every current and future gate explains itself.
3. **The store screen** (§0.5) — a list of garments. Deleting the 200-cell grid removes more code than it adds.
4. **The cutting grid** (§0.4) — the biggest new screen, and the one that saves the most time daily. Build it as a spreadsheet.
5. **Reject & rework** (§0.6) — the defect form plus the DM's queue.
6. **Job work** (§0.7) — dispatch, receive, and an overdue list.

Steps 1–3 are net deletions. Only 4–6 add screens.

Table of Contents

START HERE 0. **What changed in this release** — breaking changes, new modules, build order

WHAT CHANGED 0.0 The paging release (*current*) · 0.1–0.14 Cutting V2 / Store / Reject-Rework / Job-Work

FOUNDATION 1. Base URL and versioning 2. Authentication 3. Conventions · **3.1 Pagination** (*new*) 4. The error contract 5. The role matrix 6. Endpoint index

IDENTITY 7. Auth · 8. Users · 9. Employees

COMMERCIAL 10. Clients

INTAKE 11. Imports, the breakdown table, and release

THE SPINE 12. Barcode

MATERIALS 13. Lots and stock · 14. Suppliers · 15. The style recipe

THE FLOOR 15b. Cutting grid (*new*) · 16. Production · 17. `~~Drawers~~` → **Store** (*new*) · 17b. Inspections (*new*) · 17c. Job work (*new*)

PEOPLE 18. Attendance

MONEY 19. Wages

READS 20. Analytics · 21. Dashboards

APPENDICES 40. The mock data pack 41. Frontend call sequences 42. Quick reference card

FOUNDATION

1. Base URL and versioning

```
https://<host>/api/v1
```

Every path below is written **relative to that prefix**. Interactive docs at `/docs` and `/redoc`; liveness at `/health`, readiness at `/ready` (both unprefixd).

Group	Prefix
Authentication	<code>/auth</code>
User management	<code>/users</code>
Employees	<code>/employees</code>
Clients	<code>/clients</code>
Imports & breakdown	<code>/imports</code>
Barcode	<code>/barcode</code>
Materials	<code>/materials</code>
Material suppliers	<code>/suppliers</code>
Style recipe	<code>/styles</code>
Production	<code>/production</code>
Drawers	<code>/drawers</code>
Attendance	<code>/attendance</code>
Wages	<code>/wages</code>
Analytics	<code>/analytics</code>
Dashboards	<code>/dashboard</code>

2. Authentication

Self-issued JWT bearer tokens.

POST `/auth/login`

Request

```
{ "username": "9876543210", "password": "*****" }
```

`username` is the **phone number** — that is the login identifier. The endpoint also accepts an OAuth2 form body so Swagger's Authorize button works.

Response `200`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "role": "cutting_manager",
  "name": "Suresh Iyer",
  "user_id": "b2c3d4e5-f607-1829-3a4b-5c6d7e8f9001",
  "must_change_password": false
}
```

The token response carries the role and name. Use them to pick the landing screen immediately — you do not need a second call to `/auth/me` on login.

`must_change_password: true` → route to the change-password screen and block everything else.

Send the token on every request:

```
Authorization: Bearer <access_token>
```

Errors: `401` bad credentials · `403` inactive user.

GET `/auth/me`

Response `200`

```
{
  "id": "b2c3d4e5-f607-1829-3a4b-5c6d7e8f9001",
  "name": "Suresh Iyer",
  "phone": "9876543210",
  "email": null,
  "role": "cutting_manager",
  "is_active": true,
  "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "client_id": null
}
```

`employee_id` non-null means this login is also a person on the payroll — they can use `/attendance/me`. `client_id` non-null means an external client login, scoped to their own orders.

POST `/auth/change-password`

```
{ "current_password": ".....", "new_password": "....." }
```

Response `204` — no body. `new_password` must be **at least 8 characters**, and the service additionally refuses a password equal to the user's phone number.

3. Conventions

Aspect	Convention
IDs	UUID v4, always a string in JSON
Timestamps	ISO-8601 with timezone. Nullable fields are <code>null</code>
Dates	<code>"2026-09-09"</code>
Money	JSON <code>number</code> , two decimals
Quantities	JSON <code>number</code> , three decimals

Aspect	Convention
Enums	Production stages, designations and material categories are UPPERCASE . Roles, drawer states, run statuses and wage types are lowercase
Uploads	<code>multipart/form-data</code> . Imports use field <code>file</code> plus a <code>order_number</code> form field
Pagination	<code>limit</code> + <code>offset</code> . Paged responses carry both <code>total</code> (matching the filter) and <code>count</code> (rows in this page) — see §3.1
Empty collections	<code>[]</code> , never <code>null</code>
Synchronous	Every write is synchronous . No 202, no polling, no SSE

Three rules worth stating explicitly:

1. **Never send a production stage**. No endpoint accepts one. It is derived from the screen (fixed by the role) or inferred from the piece's history.
2. **Derived values are server-computed** — `available`, `total_cost`, `amount`, every count. Send inputs; render what comes back.
3. **Batch writes partially accept**. Read the per-item buckets, not the HTTP status.

3.1 Pagination

Twenty list endpoints are paged, and they no longer return the whole table. If your screen assumed an unbounded array, it is now showing you the first 50 rows and nothing is telling it so except `has_more`.

Why they had to be. `production_event` grows by one row per piece per stage and never shrinks. `GET /production/events` with no filter was *"serialise the whole production history"* — survivable after a few weeks of running, fatal after a year: a request that times out, a worker holding hundreds of megabytes, and a pooled database connection held for the whole of it. The pool is this service's real concurrency ceiling, so one unbounded list read degrades everything else on the box.

The ask. Two query parameters, the same two everywhere:

Param	Default	Bounds
<code>limit</code>	50	1 – 200
<code>offset</code>	0	≥ 0

The **200 cap is enforced**, not advisory: `?limit=999999` is a 422, because letting it through would reintroduce exactly the problem paging solves.

The envelope. New and newly-paged endpoints return `Page`:

```
{
  "items": [ /* ... */ ],
  "total": 3184,
  "limit": 50,
  "offset": 100,
  "count": 50,
  "has_more": true
}
```

- **`total` is the whole result set, not this page**. It is what a list screen renders as *"101–150 of 3,184"*, and it costs a second `COUNT` query — the price of a pager, and far cheaper than shipping every row.
- **`has_more` is server-derived**. `offset + len(items) < total` is exactly the off-by-one every client gets to reimplement otherwise; do not recompute it.
- **`count` is `len(items)`**. On the last page it is smaller than `limit`.

Offset, not cursor — deliberate. Offset paging is what a jump-to-page list screen needs, and that is what every one of these surfaces renders. It degrades on very deep offsets because the database still walks the skipped rows; none of these surfaces page tens of thousands deep, and a cursor API that nothing needs is a contract to maintain for nothing.

The twenty paged endpoints

Module	Endpoints
Identity	GET /users · GET /employees
Clients	GET /clients · GET /clients/styles · GET /clients/{id}/orders
Imports	GET /imports/orders
Barcode	GET /barcode/orders · GET /barcode/materials
Materials	GET /materials/leather-by-style
Production	GET /production/events · GET /production/skus · GET /production/skus/{id}/pieces
Wages	GET /wages/orders · GET /wages/styles · GET /wages/runs · GET /wages/runs/{id}/pieces · GET /wages/ledger
Analytics	GET /analytics/styles/{id}/detail

Two Phase-2 routes also take `limit / offset` and are **not** covered by any of the above: `GET /procurement/suppliers` and `GET /procurement/inventory/items`. They predate `PageParams` and declare their own `limit: int = 100, offset: int = 0` — **default 100, and no upper cap**, so `?limit=999999` is not rejected there. `/procurement/suppliers` returns `{ suppliers, count }` with no `total`. See the Procurement reference; do not assume the rules on this page apply to them.

The trap: some header counts are deliberately NOT page-scoped

`GET /production/skus/{id}/pieces` returns SKU-wide `total / done / pending / closed` alongside a 50-row page. That is not an inconsistency.

A SKU is one colour and one size of one style, and a real order runs to hundreds or thousands of garments in a single SKU. `closed` withdraws the style from the scan screen — deriving it from one page would tell a manager sitting on page 1 that a 900-piece SKU was finished. So those four counts describe the SKU.

blocked is the exception and says so. It is a page figure, and it is tagged `blocked_scope` in the response so you can label it honestly.

Do not sum header counts across pages, and do not render `count` where the screen means `total`.

Three endpoints take `limit / offset` but do NOT return `Page`

They predate the envelope and keep the shape a frontend is reading today. Quietly changing a response shape breaks a screen without breaking a test, so they migrate when their consumer is ready. **Check which one you are calling before you write the pager:**

Endpoint	Returns	What you do not get
GET /wages/runs	a bare JSON array	no envelope at all — no <code>total</code> , no <code>has_more</code> . You cannot tell a full last page from a page with more behind it except by asking for the next one.
GET /wages/ledger	<code>{ count, items }</code>	no <code>total</code> — <code>count</code> is this page only, so there is no "of 3,184" to render.
GET /wages/runs/{id}/pieces	<code>{ run_id, status, total, count, items }</code>	has <code>total</code> , but no <code>has_more</code> and no echo of <code>limit / offset</code> .

GET /imports/orders returns { total, count, items } — total is the filter match, count is this page.

Two reads carry paging fields flat, next to domain fields, rather than wrapping rows in an envelope. GET /production/skus/{id}/pieces (rows under pieces) and GET /analytics/styles/{id}/detail (rows under pieces, beside totals, stages and store) both return limit, offset, count and has_more at the top level. **Called with no paging params, limit and offset come back null and has_more is false** — and on the SKU read blocked_scope is then "sku" rather than "page". Branch on limit === null, not on the presence of the key.

Everything else in the table above returns the Page envelope exactly as shown — including GET /barcode/orders and GET /barcode/materials, which have now migrated off the old {page, page_size, total, pages, items} shape.

4. The error contract

4.1 The shape

```
{ "detail": "No drawer with code 'DRW-9999'." }
```

```
{ "detail": { "error": "sequence_blocked", "message": "..." } }
```

```
{ "detail": [ { "loc": ["body", "needs_lining"], "msg": "...", "type": "value_error" } ] }
```

Handle all three.

```
export function errorMessage(body) {
  const d = body?.detail;
  if (typeof d === "string") return d;
  if (Array.isArray(d)) return d.map(e => e.msg).join("; ");
  return d?.message ?? d?.error ?? "Something went wrong.";
}
```

A 500 carries a correlation id — **show it to the user**:

```
{ "detail": "Internal server error", "request_id": "3f9a2b71-..." }
```

4.2 Status codes

Code	Means	Do
200 / 201	OK	
204	No content	Password change, client delete
400	Bad upload	Not .xlsx, or not a valid zip container
401	Unauthenticated	Redirect to login
403	Wrong role	The message often names who can
404	Not found	Also: unknown barcode
409	State conflict	The interesting family — §4.3
410	Gone	Retired barcode, or a removed endpoint
413	File too large	Show the cap
422	Validation	Field-level, or a domain sentence
500	Server error	Show request_id

4.3 The 409 / 410 catalogue

Where	Meaning	Recovery
Edit a RELEASED style/SKU	Frozen — barcodes printed	Drive editability off <code>production_status</code>
Delete a SKU with minted pieces	Same	
Style <code>code</code> collision	Unique — a barcode caption resolves through it	Pick another
Piece scanned into the wrong drawer	The merge map is the authority	" <i>This garment belongs in DRW-xxxx</i> "
Ambiguous material lot	Several match the spec	Render the named candidates
Retire a lot with stock reserved	Reservations outstanding	Say what holds it
Lot spec collision on PATCH	Two lots a cutter cannot tell apart	Offer the existing lot
Receive with a PO mismatch	Article/colour differ	DM/MD send <code>approve_mismatch: true</code>
Confirm a spec that is incomplete	Unresolved lines	Show <code>release_blockers</code>
Replace a spec on a RELEASED style	Frozen	Use <code>POST /materials/issues</code>
Wage run overlap	Would pay twice	Link to the blocking run
Recompute a CLOSED run	Frozen	Offer Reopen with a reason
Close a run with no lines	Nobody would be paid	
Delete a client with orders	Cascade would take everything	Offer deactivate
410 — a barcode	Retired	" <i>This was deactivated</i> " — never " <i>invalid</i> "
410 — <code>/production/cutting</code> , <code>/production/scan</code>	Removed	Use <code>POST /production/log</code>
410 — <code>/analytics/overview</code> , <code>/alerts/*</code>	Removed	Use the role dashboards and <code>/dashboard/alerts</code>

5. The role matrix

MD Managing Director · DM Direct Manager · CM Cutting · LM Lining · SM Stitching · ST Store · SV Supervisor · HR · SEC Security · CL Client · VW Viewer

MD and DM are superusers and bypass every `require_roles` gate (`SUPERUSER_ROLES` in `users/deps.py`), which is why their columns are ticked throughout. (Phase 2 adds `require_exact_roles` , which the DM does not bypass — it guards BOM approval. No Phase-1 route uses it.)

MERCHANDISER and the legacy `employee` are not columns here. Both are live `UserRole` members, and **MERCHANDISER appears in no role gate anywhere** — a merchandiser token gets only what is ungated. `employee` is blocked at the router level everywhere except `auth`, `users`, `attendance` and the barcode router.

✔ granted · ✕ explicitly refused · blank = 403 from the role gate · `open` = no role gate at all (see the warning below) · `own` = scoped to the caller's own `client_id`

Identity and people

Surface	MD	DM	CM	LM	SM	ST	SV	HR	SEC	CL	VW
Login, <code>/auth/me</code> , own password	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔
List / create users	✔	✔						✔			

Surface	MD	DM	CM	LM	SM	ST	SV	HR	SEC	CL	VW
Inspections — approve / decline / responsibility	✓	✓									
Job work — read	✓	✓	✓	✓	✓	✓	✓	✓			
Job work — vendors, dispatch, receive	✓	✓									

People, money and reads

Surface	MD	DM	CM	LM	SM	ST	SV	HR	SEC	CL	VW
Write attendance (scan, manual, proxy)	✓	✓						✓	✓		
Read the roster (/attendance/today)	✓	✓	✓	✓	✓		✓	✓	✓		
Read one worker's history	✓	✓	✓	✓	✓		✓	✓	✓		
Add a daily worker	✓	✓						✓			
Correct a punch (PATCH)	✓	✓						✓	✓		
Delete a punch (DELETE)	✓	✓						✓			
Wages — READ (sheets, runs, ledger, breakdown)	✓	✓						✓			
Wages — WRITE (rates, create/recompute/reopen/close/delete a run)	✓	✓						×			
Analytics	open	open	open	open	open	open	open	open	open	open	open
/analytics/employee-rates	✓	✓						✓			
Dashboards	✓	✓	✓	✓	✓	✓	✓	✓			

Four notes, and one warning

- **HR reads payroll; HR does not write it.** Every rate write and every run operation is DM/MD. This is a deliberate separation of duties — visibility is not authority — and it was a real defect before it was fixed: a dependency named `_PAYROLL_READERS` was guarding five mutating routes.
- **HR is a materials role, not only an admin one.** HR can create and edit lots, adjust and retire them, and record receiving. That is easy to miss because HR holds no other floor permission.
- **Production stage access is a second, per-stage gate** behind the door gate — see §16.4. Passing the role gate here does not mean the stage will log.
- **STORE_MANAGER cannot log production**, cannot read the cutting grid, and does not appear on the barcode screens or the attendance roster. Store functions, dashboards, inspections, job work and materials reads only.

⚠ **open means no role gate — including CLIENT and VIEWER.** Rows marked `open` are guarded by `get_current_user` alone: any authenticated token reaches them. A `CLIENT` token is narrowed to its own `client_id` by `client_scope` on the client, analytics and production reads, so it sees only its own orders — **but a VIEWER token is not narrowed at all** and reads the whole factory on those routes. Two reads that look open are not: `/employees` and `/employees/{id}` refuse CLIENT and VIEWER inside the handler, and `/attendance/history` refuses anyone outside `_ATTENDANCE_READERS`.

6. Endpoint index

154 endpoints under `/api/v1`, excluding the Procurement module (Phase 2 — 65 endpoints, its own reference) and `/chat` (2). The app serves **224 routes** in total: those 154, plus Procurement's 65, plus `/chat` and `/chat/stream`, plus `/`, `/health` and `/ready` outside `/api/v1`.

New in this release: **Cutting (8), Store (4), Inspections (6), Job work (5)**, plus 3 new material reads, 2 production corrections, 2 attendance corrections and the single-employee read. **Drawers (9) are withdrawn. Four routes were removed on 2026-09-19** — `POST /users/clients` (\$8.3), `~~ GET /attendance/me/status` `~~` (\$18.8), `~~ GET / PATCH /attendance/config` `~~` (\$18.9). They are struck out below and each section says why.

This index is checked against the running app. `python scripts/make_postman_per_service.py` regenerates `postman/` from `/openapi.json`; if a path here is not in that folder, one of the two is wrong.

Identity (10)

Method	Path	Roles
POST	<code>/auth/login</code>	public
GET	<code>/auth/me</code>	any
POST	<code>/auth/change-password</code>	any
GET	<code>/users</code>	DM HR MD
POST	<code>/users</code>	DM HR MD
~~ POST ~~	~~ <code>/users/clients</code> ~~	REMOVED — \$8.3
GET	<code>/employees</code>	internal
POST	<code>/employees</code>	DM HR MD
GET	<code>/employees/{id}</code>	internal
PATCH	<code>/employees/{id}</code>	DM HR MD
DELETE	<code>/employees/{id}</code>	DM MD

Clients (8)

`GET / POST /clients` · `GET /clients/styles` · `GET / PATCH / DELETE /clients/{id}` · `GET / POST /clients/{id}/orders`

Imports (9)

`POST /imports/preview` · `POST /imports/commit` · `GET /imports/orders` · `GET /imports/breakdown/{order_number}` · `PATCH /imports/breakdown/skus/{id}` · `PATCH /imports/breakdown/styles/{id}` · `DELETE /imports/breakdown/skus/{id}` · `POST /imports/breakdown/{order_number}/cancel` · `POST /imports/breakdown/{order_number}/release`

Barcode (10)

`GET /barcode/resolve` · `POST /barcode/print` · `PATCH /employees/{id}/barcode` · `GET /barcode/materials` · `GET /barcode/orders` · `GET /barcode/orders/by-number/{n}` · `GET /barcode/orders/{id}/skus` · `GET /barcode/orders/{id}/analytics` · `GET /barcode/orders/{id}/barcodes` · `GET /barcode/detail`

Materials (24)

Lots (7): POST / GET /materials/lots · GET / PATCH / DELETE /materials/lots/{id} · PATCH /materials/lots/{id}/adjust · GET /materials/spec **Stock & receiving (2):** GET /materials/stock · POST /materials/receive (both now take an optional sheets[] for LEATHER — §0.8) **Consumption reads (3, new):** GET /materials/lots/{id}/history · GET /materials/leather-by-style · GET /materials/pieces/{id}/consumption **Issues (1):** POST /materials/issues **Suppliers (3):** POST /suppliers/orders · PATCH /suppliers/orders/{id} · PATCH /suppliers/orders/{id}/spec **Recipe (8):** GET / PUT /styles/{id}/material-spec · POST /styles/{id}/material-spec/lines · PATCH / DELETE /styles/{id}/material-spec/lines/{lid} · POST /styles/{id}/material-spec/confirm · POST /styles/{id}/material-spec/copy-from · GET /styles/{id}/material-spec/requirement

Cutting (8) — NEW · §0.4

GET /cutting/grid · POST /cutting/rows/generate · PATCH /cutting/rows/{id} · POST /cutting/rows/{id}/approve · POST /cutting/rows/{id}/reopen · POST /cutting/rows/{id}/sheets · PATCH / DELETE /cutting/rows/{id}/sheets/{sid} Roles: CUTTING_MANAGER, DM, MD.

Production (11)

GET /production/operations · GET /production/skus · GET /production/events · GET /production/styles/{id}/progress · GET /production/skus/{id}/pieces · GET /production/piece-state · POST /production/log · PATCH /production/events/{id}/reassign (new) · DELETE /production/events/{id} (new) · POST /production/cutting (410) · POST /production/scan (410)

Store (4) — NEW, replaces Drawers · §0.5

POST /store/scan · POST /store/send · GET /store/pieces · GET /store/pieces/{piece_code} Scan/send: STORE_MANAGER, STITCHING_MANAGER, DM, MD. Read: all floor roles + HR.

Inspections (6) — NEW · §0.6

POST /inspections · POST /inspections/{id}/approve · POST /inspections/{id}/decline · GET /inspections · GET /inspections/responsibility · GET /inspections/pieces/{code} Raise: all managers + HR. Approve/decline: DM, MD only.

Job work (5) — NEW · §0.7

POST / GET /jobwork/vendors · POST /jobwork/dispatch · POST /jobwork/{id}/receive · GET /jobwork Dispatch/receive: DM, MD. Read: all floor roles + HR.

~~Drawers (9)~~ — WITHDRAWN, all routes 404 · §0.2

Replaced by **Store** above. Old DRAWER barcodes still resolve (a printed sticker must not 404) but return next_expected_scan: null and frozen history.

Attendance (11)

POST /attendance/scan-check-in · POST /attendance/check-in · POST /attendance/check-out · POST /attendance/proxy/check-in · POST /attendance/proxy/check-out · POST /attendance/daily-workers · GET /attendance/me · GET /attendance/history · GET /attendance/today · PATCH /attendance/{id} · DELETE /attendance/{id} Removed 2026-09-19: ~~ GET /attendance/me/status ~~ (§18.8) · ~~ GET / PATCH /attendance/config ~~ (§18.9). The shift policy they exposed still applies on every punch — only the API for it is gone.

Wages (16)

GET /wages/orders · GET /wages/styles · GET /wages/rate-sheet · GET /wages/rate-history · POST /wages/rates · POST /wages/rates/bulk · GET / POST /wages/runs · GET /wages/runs/{id} · DELETE /wages/runs/{id} · POST /wages/runs/{id}/recompute · POST /wages/runs/{id}/reopen · POST /wages/runs/{id}/close · GET /wages/runs/{id}/breakdown · GET /wages/runs/{id}/pieces · GET /wages/ledger

Analytics (10)

All GET, all under /analytics: /explorer · /orders/{order_id}/tree · /styles/{style_id}/detail · /pieces/detail · /pieces/{piece_code}/story · /consumption · /employee-rates **410 Gone (deliberate, not deleted):** /overview · /alerts/stage-spread · /alerts/freight-risk — use the role dashboards and GET /dashboard/alerts.

Dashboard (22)

All GET, all under /dashboard. Every one is a read; the module owns no table.

Screen	Paths
Cutting (4)	/cutting · /cutting/consumption · /cutting/employees/{employee_id} · /cutting/pieces/{piece_code}
Lining (4)	/lining · /lining/consumption · /lining/employees/{employee_id} · /lining/pieces/{piece_code}
Stitching (3)	/stitching · /stitching/employees/{employee_id} · /stitching/pieces/{piece_code}
Store (5)	/store · /store/traceability · /store/pieces/{piece_code} · /store/drawers/{drawer_id} · /store/drawers/{drawer_id}/movement
Direct Manager (4)	/direct-manager · /direct-manager/orders/{order_id} · /direct-manager/styles/{style_id} · /direct-manager/pieces/{piece_code}
Generic piece trace (1)	/pieces/{piece_code}
Alerts (1)	/alerts

The five pieces/{piece_code} paths are aliases of one trace, one per screen, so each dashboard can link to a garment without leaving its own namespace. They differ in framing, not in the underlying piece.

The two store/drawers/* paths are the only drawer-shaped routes left. They read frozen history for audit. They are not live state, and there is no drawer pool behind them — see §17.

IDENTITY

7. Auth

Covered in §2 — POST /auth/login, GET /auth/me, POST /auth/change-password.

8. Users

Router: `app/modules/users/router.py`

8.1 GET /users

List every login. Roles: DM · HR · MD · Query: `active_only` (default `true`)

Response `200`

```
[
  {
    "id": "a1b2c3d4-e5f6-0718-293a-4b5c6d7e8f90",
    "name": "Tanveer Ahmed",
    "phone": "9000000001",
    "email": "tanveer@ptexports.com",
    "role": "managing_director",
    "is_active": true,
    "employee_id": null,
    "client_id": null
  },
  {
    "id": "b2c3d4e5-f607-1829-3a4b-5c6d7e8f9001",
    "name": "Suresh Iyer",
    "phone": "9876543210",
    "email": null,
    "role": "cutting_manager",
    "is_active": true,
    "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
    "client_id": null
  }
]
```

8.2 POST /users

Create a staff / manager / viewer login. Roles: DM · HR · MD

The router admits all three; **which role may be granted is decided in the service against the caller's own authority.** HR cannot mint an MD.

Request

```
{
  "name": "Priya Nair",
  "phone": "9812345678",
  "email": "priya@ptexports.com",
  "role": "stitching_manager",
  "password": ".....",
  "employee_id": null
}
```

`role` defaults to `viewer`. `employee_id` links this login to an existing payroll record.

Response `201` — a `UserRead` object as in 8.1.

Errors: `403` caller may not grant that role · `409` phone or email already in use · `422`

8.3 ~~ POST /users/clients ~~ — REMOVED 2026-09-19

The route is unrouted. It 404s. Do not build against it.

It provisioned a CLIENT-role login bound to a `client_id`, so a buyer could log in and watch their own orders.

Why it went. Phase 1 is the factory floor's system of record — order sheet in, breakdown, barcode, cut, stitch, wages. The buyer is not a user of it: they send an order sheet and they receive a BOM, both by hand, and the one thing they approve (the costing) is approved off-system before production starts. A client login had nothing to do inside the app that a person was not already doing for them outside it.

Why removing it was worth doing rather than leaving it unused. It was the *only* way a `CLIENT` login could ever be created. Every `if user.role == UserRole.CLIENT` branch in `clients/`, `analytics/`, `dashboard/`, `production/` and `bom/` is a cross-tenant scoping check, and each one is a place a buyer could be shown another buyer's styles, prices or order book if the scoping were ever got wrong. With no door to mint the role, that surface cannot be reached at all.

`POST /clients` is a different thing and still exists (§10). It creates the client *record* — the buyer the orders hang off. It mints no login.

Putting it back is meant to be uncommenting a route, not rewriting a feature: `UserService.create_client_user()` and `schemas.ClientUserCreate` are left in place, and the `CLIENT`-role branches stay where they are against a Phase-2 client portal.

9. Employees

Router: `app/modules/employees/router.py`

9.1 GET /employees

The roster. Roles: every internal role. **CLIENT and VIEWER are refused (403)** — the row carries phone and email.

Query: `active_only` (default `true`)

Response 200 — for **HR / DM / MD** (includes salary)

```
[
  {
    "id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
    "name": "Ramesh Kumar",
    "designation": "CUTTER",
    "wage_type": "piece_rate",
    "is_active": true,
    "phone": null,
    "email": null,
    "role": null,
    "employee_barcode": "EMP-000123",
    "monthly_salary": null
  },
  {
    "id": "d4e5f607-1829-3a4b-5c6d-7e8f9001a2b3",
    "name": "Farid Ansari",
    "designation": "LINE_TAILOR",
    "wage_type": "monthly",
    "is_active": true,
    "phone": "9700011122",
    "email": null,
    "role": null,
    "employee_barcode": "EMP-000124",
    "monthly_salary": 18000.0
  }
]
```

For every other internal role, the identical shape **without** `monthly_salary` .

Field	Meaning
<code>designation</code>	Always uppercase. Drives the production skill gate
<code>wage_type</code>	<code>piece_rate</code> <code>monthly</code> . Says nothing about system access
<code>role</code>	Non-null only when this employee also holds a staff login
<code>employee_barcode</code>	The active card. null when retired or never issued — a retired code resolves 410, so showing it would invite a failing scan

9.2 POST /employees

Create a person on the payroll. Roles: DM · HR · MD

Request — a plain worker (the common case)

```
{ "name": "Ramesh Kumar", "designation": "CUTTER", "wage_type": "piece_rate" }
```

No phone, no email, no password, no role. **No** `app_user` row is created.

Request — staff, with a login

```
{
  "name": "Priya Nair",
  "designation": "SUPERVISOR",
  "wage_type": "monthly",
  "monthly_salary": 26000,
  "phone": "9812345678",
  "password": ".....",
  "role": "supervisor"
}
```

Response 201

```
{
  "id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "name": "Ramesh Kumar",
  "designation": "CUTTER",
  "wage_type": "piece_rate",
  "is_active": true,
  "phone": null,
  "email": null,
  "role": null,
  "employee_barcode": "EMP-000123",
  "monthly_salary": null,
  "user_created": false,
  "login_phone": null
}
```

employee_barcode is minted in the same transaction — print the card immediately.

Validation rules (all 422)

Rule	Message
<code>role: "employee"</code>	<i>"The 'employee' role is not assignable — shop-floor workers have no login."</i>
<code>role</code> is DM or MD	<i>"...created via user-creation, not the employee service."</i>
<code>role</code> outside the staff set	Names the permitted roles
Staff role without phone/password	<i>"phone, password required when creating a staff login"</i>
A password with no role	<i>"password is only accepted when a staff login is created — workers do not log in"</i>

Permitted staff roles: `hr` · `supervisor` · `cutting_manager` · `lining_manager` · `stitching_manager` · `security` · `merchandiser`

9.3 GET /employees/{employee_id}

One worker. Roles: every internal role. **CLIENT and VIEWER are refused (403)** — same gate as the roster, and for the same reason: the row carries phone and email.

This is what the edit screen and the worker card open on. Before it existed the only way to show one person was to fetch the whole roster and filter in the browser, which meant every screen that showed one worker downloaded every worker's phone number.

Response 200 — for HR / DM / MD (includes salary)

```
{
  "id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "name": "Ramesh Kumar",
  "designation": "CUTTER",
  "wage_type": "piece_rate",
  "is_active": true,
  "phone": null,
  "email": null,
  "role": null,
  "employee_barcode": "EMP-000123",
  "monthly_salary": null
}
```

For every other internal role the shape is the same **without** `monthly_salary` — a supervisor reading the floor roster has no business seeing pay.

`is_active: false` is a **normal 200**, not a 404. A worker who has left is still readable — their production events and wage lines reference them, and a name that stops resolving turns last quarter's payroll into an unanswerable question. Only the **card** stops working (`GET /barcode/resolve` → **410 Gone**).

Errors: 404 no such employee · 403 CLIENT or VIEWER.

9.4 PATCH /employees/{employee_id}

Edit an employee. Roles: DM · HR · MD. All fields optional.

Request

```
{ "designation": "SHELL_TAILOR", "monthly_salary": 19500 }
```

Granting a login later requires **both** `role` and `password` together — either alone is a 422.

Response 200 — an `EmployeeRead` with `employee_barcode` and `role` read back from the login table.

9.5 DELETE /employees/{employee_id}

Soft-delete + retire the card. Roles: DM · MD only — **HR can edit but not remove.**

Response 200

```
{
  "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "active": false,
  "history_preserved": true
}
```

Deleting somebody already inactive is **not an error** — it returns 200 with `"already_inactive": true` in place of `history_preserved`, so a double-click does not produce a scary red box for a no-op.

The employee row, every production event and every wage line stay intact. Scanning the retired card returns **410 Gone** — a distinct answer from 404, which means the code never existed at all.

COMMERCIAL

10. Clients

Router: `app/modules/clients/router.py`

Route order is load-bearing: `GET /clients/styles` is declared **before** `GET /clients/{client_id}`, or the literal path becomes unreachable.

10.1 GET /clients

Roles: any authenticated. **A CLIENT login sees only its own row.**

Query: `include_inactive` (default `false`)

Response 200

```
[
  {
    "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "name": "BOGGI MILANO",
    "country": "Italy",
    "code": "BOG",
    "currency": "EUR",
    "default_size_system": "eu",
    "brand": "BOGGI",
    "label": "BOGGI MILANO",
    "contact_email": "purchasing@boggi.example",
    "contact_phone": "+390212345678",
    "address": "Via Durini 28, Milano",
    "is_active": true
  }
]
```

Deactivated clients are hidden by default — that is what deactivation is for.

10.2 POST /clients

Create a client and its first order in one call. Roles: DM · MD

Request

```
{ "name": "BOGGI MILANO", "country": "Italy", "order_number": "BOG-SS27-001" }
```

Response 201

```
{
  "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "name": "BOGGI MILANO",
  "country": "Italy",
  "code": "BOG",
  "currency": "EUR",
  "default_size_system": "eu",
  "order_number": "BOG-SS27-001",
  "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899"
}
```

10.3 GET /clients/{client_id}

Roles: any authenticated; **a CLIENT may read only itself (403 otherwise).** Response as in 10.1.

10.4 PATCH /clients/{client_id}

Roles: DM · MD. **Partial** — only fields present are written, so a form posting one field cannot blank the rest.

This is also the deactivate door:

```
{ "is_active": false }
```

Errors: 409 — code collision (unique across clients) · 404

10.5 DELETE /clients/{client_id}

Roles: DM · MD. **Response** 204 .

Works only on a client with **no orders**. A client *with* orders is a **409** — the cascade would take their orders, styles, SKUs and every piece, production event and wage line. **The error names the exact call to make instead** (the PATCH above).

10.6 GET /clients/{client_id}/orders

Roles: any authenticated; a CLIENT may view only their own (403 otherwise).

Response 200

```
[
  {
    "id": "3a4b5c6d-7e8f-9001-1223-3445566778899",
    "order_number": "BOG-SS27-001",
    "order_date": "2026-06-14",
    "delivery_deadline": "2026-11-30",
    "sea_cutoff_date": "2026-10-05",
    "ship_mode": "sea",
    "currency": "EUR",
    "agent": "MEDITEX",
    "line": "MAIN",
    "styles": [
      {
        "id": "4b5c6d7e-8f90-0112-2334-4556677889900",
        "name": "CLERMONT",
        "article": "A-4471",
        "code": "BOG-CLERMONT",
        "thickness": "0.7",
        "season": "SS27",
        "unit_price": 118.00,
        "currency": "EUR",
        "production_status": "DRAFT",
        "needs_lining": null
      }
    ]
  }
]
```

sea_cutoff_date is the last day to make sea freight. Missing it means air freight — the single biggest margin event in the business. It drives the freight-risk alert.

10.7 POST /clients/{client_id}/orders

Roles: DM · MD

Request

```
{
  "order_number": "BOG-AW27-002",
  "order_date": "2026-09-01",
  "delivery_deadline": "2027-02-28",
  "sea_cutoff_date": "2027-01-05",
  "ship_mode": "sea",
  "currency": "EUR",
  "agent": "MEDITEX",
  "line": "OUTLET"
}
```

Response 201 — the order with `styles: []`.

Errors: 409 — `order_number` is unique across the whole system.

10.8 GET /clients/styles

The style picker. Roles: any authenticated.

Query: `order_number` · `client_id`

A CLIENT caller is pinned to their own `client_id` regardless of what they pass, and it is combined (AND) with any `order_number` — so a borrowed order number cannot read another client's styles.

Response 200

```
[
  {
    "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900",
    "code": "BOG-CLERMONT",
    "name": "CLERMONT",
    "article": "A-4471",
    "order_number": "BOG-SS27-001",
    "client_name": "BOGGI MILANO",
    "qty_ordered": 60
  }
]
```

INTAKE

11. Imports, the breakdown table, and release

Router: `app/modules/imports/router.py` · **Roles for every endpoint:** DM · MD

11.1 POST /imports/preview

Dry-run an breakdown workbook. Writes nothing.

Request: `multipart/form-data` — field `order_number` (form) **and** field `file` (the `.xlsx`)

The order number is validated **first**: a number that matches no client order is a **404**, and the file is never parsed.

Response 200

```
{
  "clients": 1,
  "orders": 1,
  "styles": 2,
  "skus": 5,
  "pieces": 100,
  "warnings": [
    "row 14: size '50 ' had trailing whitespace - normalised",
    "row 22: colour code missing, used colour name 'COGNAC'",
    "style CARNABY: no thickness in the sheet"
  ],
  "styles_detail": [
    { "name": "CLERMONT", "article": "A-4471", "skus": 3, "qty": 60 },
    { "name": "CARNABY", "article": "A-4482", "skus": 2, "qty": 40 }
  ]
}
```

Errors

Cod e	When
400	Not <code>.xlsx</code> / <code>.xslm</code> , or not a valid zip container (an <code>.xlsx</code> is a zip; anything renamed is rejected before it reaches the parser)
404	Order number not found
413	Over <code>max_upload_mb</code> — streamed, aborts mid-upload

11.2 POST /imports/commit

Parse and write SKUs into the named order. Idempotent.

Request: identical to 11.1.

Response 200

```
{
  "summary": {
    "clients": 1, "orders": 1, "styles": 2, "skus": 5, "pieces": 100,
    "warnings": []
  },
  "written": {
    "styles_created": 2,
    "styles_updated": 0,
    "skus_created": 5,
    "skus_updated": 0,
    "pieces_minted": 0,
    "release_required": true,
    "order_number": "BOG-SS27-001",
    "breakdown_url": "/api/v1/imports/breakdown/BOG-SS27-001"
  }
}
```

`pieces_minted: 0` and `release_required: true` are the whole point. Commit writes **DRAFT** styles and SKUs. No pieces, no barcodes, no drawer merges. Nothing is irreversible yet.

Next call: `GET /imports/breakdown/{order_number}`.

The order number in this response is a **receipt, not the only copy**. The order is a permanent row, and every order is listed at `GET /imports/orders`. Losing this response loses nothing.

11.3 GET /imports/orders

The order index — the screen the breakdown opens from.

Query: `q` (order number **or** client name, case-insensitive contains) · `client_id` · `status` · `has_breakdown` · `limit` (50) · `offset`

Response 200

```
{
  "total": 12,
  "count": 2,
  "orders": [
    {
      "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899",
      "order_number": "BOG-SS27-001",
      "client_id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "client_name": "BOGGI MILANO",
      "order_date": "2026-06-14",
      "delivery_deadline": "2026-11-30",
      "breakdown_status": "PARTIALLY_RELEASED",
      "styles": 2,
      "styles_released": 1,
      "pieces_minted": 40,
      "breakdown_url": "/api/v1/imports/breakdown/BOG-SS27-001",
      "last_activity_at": "2026-09-09T10:14:02.118000+00:00"
    },
    {
      "order_id": "5c6d7e8f-9001-1223-3445-566778899001",
      "order_number": "GGZ-AW26-007",
      "client_id": "8d0f7780-8536-51ef-a55c-f18fd2fa1bf8",
      "client_name": "GGZ APPAREL",
      "order_date": "2026-03-02",
      "delivery_deadline": "2026-09-30",
      "breakdown_status": "NOT_UPLOADED",
      "styles": 0,
      "styles_released": 0,
      "pieces_minted": 0,
      "breakdown_url": "/api/v1/imports/breakdown/GGZ-AW26-007",
      "last_activity_at": "2026-03-02T08:00:00.000000+00:00"
    }
  ]
}
```

breakdown_status — rolled up from the order's styles:

Value	Meaning
NOT_UPLOADED	The order exists; no breakdown committed yet
DRAFT	Sheet uploaded, nothing released, nothing minted
PARTIALLY_RELEASED	Some styles in production, some still editable
RELEASED	Every live style released
CANCELLED	Every style was cancelled

Ordered most-recently-worked-on first — the newest of the order's creation and its last-written style. `order_date` is the client's date and says nothing about what the factory is handling now.

Errors: `422` — `status` outside the five values (the message lists them).

11.4 GET `/imports/breakdown/{order_number}`

The breakdown table — the screen between upload and production.

Response `200`

```

{
  "order_number": "BOG-SS27-001",
  "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899",
  "client_name": "BOGGI MILANO",
  "styles": [
    {
      "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900",
      "name": "CLERMONT",
      "article": "A-4471",
      "code": "BOG-CLERMONT",
      "gender": "MENS",
      "label": "BOGGI MILANO",
      "thickness": "0.7",
      "season": "SS27",
      "customer_ref": "CR1-02F5-PL02",
      "internal_ref": "A32073-44",
      "unit_price": 118.00,
      "currency": "EUR",
      "production_status": "DRAFT",
      "released_at": null,
      "released_by": null,
      "needs_lining": null,
      "lining_reason": "not declared - release will reject this style",
      "minted_pieces": 0,
      "material_spec_confirmed_at": "2026-09-08T16:20:11.004000+00:00",
      "release_blockers": [],
      "skus": [
        {
          "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
          "code": "BOG-CLERMONT-BLK-50",
          "color_code": "BLK",
          "color_name": "BLACK",
          "size": "50",
          "qty_ordered": 20,
          "knit_color": null,
          "nylon_color": null,
          "minted_pieces": 0
        }
      ]
    },
    {
      "style_id": "7e8f9001-1223-3445-5667-788990011223",
      "name": "CARNABY",
      "article": "A-4482",
      "code": "BOG-CARNABY",
      "thickness": "0.9",
      "production_status": "RELEASED",
      "released_at": "2026-09-09T09:02:44.881000+00:00",
      "released_by": "Tanveer Ahmed",
      "needs_lining": true,
      "lining_reason": null,
      "minted_pieces": 40,
      "release_blockers": [],
      "skus": [ /* ... read-only ... */ ]
    }
  ]
}

```

Field	Meaning
production_status	DRAFT rows are editable. RELEASED rows are read-only

Field	Meaning
needs_lining	null = not declared. Release will reject it
lining_reason	Why the effective verdict differs from the flag
minted_pieces	Real barcoded garments behind this style/SKU
release_blockers	What stands between this style and release — e.g. an unconfirmed recipe

11.5 PATCH /imports/breakdown/skus/{sku_id}

Correct one DRAFT line — the SKU, its parent style, or both in one call.

Request

```
{
  "qty_ordered": 22,
  "color_name": "NERO",
  "color_code": "BLK",
  "size": "50",
  "knit_color": null,
  "nylon_color": null,
  "style": {
    "article": "A-4471-B",
    "thickness": "0.75",
    "unit_price": 121.50,
    "needs_lining": true
  }
}
```

Send `style: {...}` alongside the SKU fields and both move in ONE transaction. Two round trips meant a 200 on the SKU and a 409 on the style left the screen half-saved with no way to tell which half.

Nested style accepts: `name` · `article` · `code` · `gender` · `label` · `thickness` · `season` · `customer_ref` · `internal_ref` · `unit_price` · `currency` · `needs_lining`

Response 200 — the updated style block with its SKUs.

Errors

Code	When
409	The style is RELEASED. <i>"To make MORE of a released style, raise the quantity on a new upload and release again — release tops up"</i>
409	Style <code>code</code> collision
404 / 422	

11.6 PATCH /imports/breakdown/styles/{style_id}

The style-only door — for the style header, which has no SKU to hang off. Same body as the nested `style` object above, same DRAFT-only rule, same audit row.

11.7 DELETE /imports/breakdown/skus/{sku_id}

Drop a line the sheet should not have had. DRAFT and zero minted pieces only.

Response 200

```
{ "sku_id": "6d7e8f90-...", "deleted": true }
```

11.8 POST /imports/breakdown/{order_number}/cancel

Withdraw DRAFT styles so they stop appearing on the release screen.

Request

```
{ "style_ids": ["7e8f9001-1223-3445-5667-788990011223"] }
```

Response 200

```
{ "cancelled": 1, "style_ids": ["7e8f9001-..."], "rejected": [] }
```

11.9 POST /imports/breakdown/{order_number}/release

★ THE MINT

This is the single most consequential call in the system. It creates Piece rows, per-piece barcodes and drawer merges, atomically. It cannot be undone.

Three request shapes

A — the contract. Each style carries its own answer:

```
{
  "styles": [
    { "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900", "needs_lining": true },
    { "style_id": "7e8f9001-1223-3445-5667-788990011223", "needs_lining": false }
  ],
  "grow_drawer_pool": true
}
```

B — broadcast. One human answer covering the list. **Still a declaration.**

```
{
  "style_ids": ["4b5c6d7e-...", "7e8f9001-..."],
  "needs_lining": true
}
```

C — deprecated. `style_ids` without a top-level answer falls back to *inferring* the lining requirement from the style name. Every style released this way comes back with `lining_declared: false`, and the message names them.

Field	Type	Notes
<code>styles[]</code>	array	<code>{style_id, needs_lining}</code> — preferred
<code>style_ids[]</code>	array	Deprecated alone; fine with a top-level answer
<code>needs_lining</code>	bool?	Broadcast. A per-style answer overrides it
<code>grow_drawer_pool</code>	bool	Default <code>false</code> . Mints the drawer shortfall in the same call

Response 201

```
{
  "order_number": "BOG-SS27-001",
  "released": [
    {
      "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900",
      "name": "CLERMONT",
      "needs_lining": true,
      "lining_declared": true,
      "pieces_minted": 60,
      "barcodes_minted": 60,
      "drawers_merged": 60,
      "released_at": "2026-09-09T11:04:22.118000+00:00",
      "released_by": "Tanveer Ahmed"
    }
  ],
  "rejected": [
    {
      "style_id": "7e8f9001-1223-3445-5667-788990011223",
      "name": "CARNABY",
      "reason": "already RELEASED on 2026-09-09T09:02:44Z - release tops up quantity only"
    }
  ],
  "minted": {
    "pieces": 60,
    "barcodes": 60,
    "drawers_merged": 57,
    "pieces_waiting_for_drawer": 3
  },
  "drawer_pool": { "size": 200, "free": 0, "shortfall": 3 },
  "message": "Released 1 style, 60 pieces minted. 3 pieces have no drawer - grow the pool or wait for one to free up."
}
```

Errors

Cod e	When
422	A style with no lining answer. The message names every unanswered style and explains that the answer cannot be changed after release
422	<code>styles</code> and <code>style_ids</code> both sent — two lists with no defined precedence
422	Neither sent
422	Any unknown field — the body uses <code>extra="forbid"</code> , so a misplaced <code>needs_lining</code> is named rather than silently dropped
404	Order number unknown

Frontend rules

1. **Never default the lining radio.** `null` is a real state and the server rejects it.
2. **Read `minted`, not the HTTP status.** Partial accept: an already-released style lands in `rejected`; the rest still release.
3. **Surface `pieces_waiting_for_drawer` prominently.** Those pieces have barcodes but nowhere to be stored — they cannot pass the merge gate.
4. **Warn that release is irreversible** before the click.

THE SPINE

12. Barcode

Router: `app/modules/barcode/router.py`

12.1 GET /barcode/resolve

Every scan's front door. Roles: any authenticated — deliberately **not** behind `block_employees`, because the attendance screen needs it.

Query: `code` (required)

Response 200 — a PIECE

```
{
  "code": "PC-004821",
  "type": "PIECE",
  "active": true,
  "caption": "CLERMONT · BLACK · 50 · #017",
  "is_alias": false,
  "piece": {
    "piece_id": "8f900112-2334-4556-6778-899001122334",
    "piece_code": "KJ2451-CLERMONT-BLK-50-017",
    "serial": "017",
    "seq": 17,
    "article": "A-4471",
    "style_name": "CLERMONT",
    "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900",
    "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
    "sku_code": "BOG-CLERMONT-BLK-50",
    "colour": "BLACK",
    "size": "50",
    "order_number": "BOG-SS27-001",
    "client_name": "BOGGI MILANO",
    "needs_lining": true,
    "current_stage": "PASTING",
    "current_stage_label": "Pasting",
    "display_stage": "STORE",
    "display_label": "In store",
    "in_store": true,
    "drawer": { "drawer_id": "90011223-3445-5667-7889-900112233445",
      "code": "DRW-0042", "state": "received", "holding": "HOLDING BOTH" },
    "consumption": { "leather_lot": "A-338", "dcm": 34.5 }
  },
  "next_expected_scan": "DRAWER",
  "next_stage": "LINE_STITCHING",
  "next_stage_label": "Line stitching",
  "next_stage_blocked_reason": "the drawer has not been sent"
}
```

Response 200 — an EMPLOYEE


```
{
  "code": "EMP-000123",
  "type": "EMPLOYEE",
  "active": true,
  "caption": "Ramesh Kumar · CUTTER",
  "is_alias": false,
  "employee": {
    "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
    "name": "Ramesh Kumar",
    "designation": "CUTTER",
    "wage_type": "piece_rate",
    "is_active": true,
    "present_today": true,
    "checked_in_at": "2026-09-09T03:32:10.000000+00:00"
  },
  "next_expected_scan": "PIECE",
  "next_stage": null,
  "next_stage_label": null,
  "next_stage_blocked_reason": null
}
```

Response 200 — a **DRAWER** carries a `drawer` block; a **LOT** carries a `lot` block with the three stock numbers.

Errors

Code	Meaning	Say
404	Unknown code	"Unknown barcode."
410	Retired	" <i>This barcode was deactivated. The record and history are intact; issue a new label.</i> "

Three fields worth building around

- **is_alias: true** — a legacy long code. The scan works; nudge the operator to reprint with the compact code.
- **next_expected_scan** — "PIECE" / "DRAWER" / null . Answered for **every** type, including EMPLOYEE (the first scan of every workflow). Drive the wizard off it.
- **next_stage** + **next_stage_blocked_reason** — the stage is **still reported** when blocked. Show where the piece is going **and** what is holding it there.

410 applies to every barcode type, not only employee cards.

12.2 GET /barcode/detail

Full detail for one barcode, for click-through from the history list. **Roles:** floor + office readers.

Same body as 12.1, tenancy-scoped for CLIENT logins.

12.3 POST /barcode/print

Code128-ready label payloads. Roles: DM · MD · Cutting · Stitching · HR

Request — exactly one source:

```
{ "sku_id": "6d7e8f90-0112-2334-4556-677889900112" }
```

```
{ "codes": ["PC-004821", "PC-004822"] }
```

```
{ "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899" }
```

Response 200

```
{
  "labels": [
    {
      "code": "PC-004821",
      "symbology": "code128",
      "caption": "CLERMONT · BLACK · 50 · #017",
      "known": true,
      "details": {
        "order_number": "BOG-SS27-001",
        "article": "A-4471",
        "style": "CLERMONT",
        "colour": "BLACK",
        "size": "50",
        "serial": "017",
        "piece_code": "KJ2451-CLERMONT-BLK-50-017"
      },
      "label_line": "BOG-SS27-001 · A-4471 · CLERMONT · BLACK · 50 · #017"
    }
  ]
}
```

Encode `code` (the compact id). **Typeset** `label_line` underneath. `details` is null for non-piece labels — they name no garment.

Errors: 422 — none of the three sources given.

12.4 PATCH /employees/{employee_id}/barcode

Reissue or deactivate an employee card. Roles: DM · MD · HR

Request

```
{ "action": "reissue" }
```

`action` ∈ `reissue` · `deactivate`

Response 200

```
{
  "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "employee_barcode": "EMP-000456",
  "active": true,
  "history_preserved": true
}
```

- **Reissue** — retires the old code, mints a new one. Print the new card.
- **Deactivate** — retires the code. `employee_barcode` echoes the retired code, `active: false`.

`history_preserved: true` is always true. The employee row, every production event and every wage line stay intact.

You delete the scannable code, never the person or their record.

12.5 GET /barcode/materials

The material-barcode screen — reprint a lost lot label. **Roles:** floor + office readers.

Query: `category ( LEATHER | LINING | ACCESSORY ) · active_only (default true)`

Response 200

```
{
  "count": 2,
  "lots": [
    {
      "code": "LOT-000338",
      "status": "active",
      "label_line": "LEATHER · SHEEP GLASS · BLACK · 0.7 · 3400 dcm",
      "lot_id": "a1338000-0000-0000-0000-000000000338",
      "category": "LEATHER", "subtype": null,
      "article": "SHEEP GLASS", "colour": "BLACK", "thickness": "0.7",
      "on_hand": 3400.0, "uom": "dcm"
    },
    {
      "code": "LOT-000112",
      "status": "retired",
      "label_line": "ACCESSORY · YKK ZIP #5 60CM · BLACK · 0 pcs",
      "lot_id": "a1112000-0000-0000-0000-000000000112",
      "category": "ACCESSORY", "subtype": "ZIP",
      "article": "YKK ZIP #5 60CM", "colour": "BLACK", "thickness": null,
      "on_hand": 0.0, "uom": "pcs"
    }
  ]
}
```

A row with `status: "retired"` should be greyed and not printed.

A UI note from the source: the drawer screen's label reads *"bucket barcode"* and should read *"drawer barcode"*. The backend has only ever called it DRAWER — there is no `bucket` anywhere in the API. That rename is frontend-only.

12.6 GET /barcode/orders

The order picker for barcode screens. Roles: floor + office readers.

Response 200

```
[
  {
    "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899",
    "order_number": "BOG-SS27-001",
    "client_name": "BOGGI MILANO",
    "minted": 60,
    "first_generated_at": "2026-09-09T11:04:22.118000+00:00",
    "last_generated_at": "2026-09-09T11:04:31.442000+00:00"
  }
]
```

Only orders that have generated barcodes appear.

12.7 GET /barcode/orders/by-number/{order_number}

Resolve a human order number to its picker row. Same shape as one element of 12.6.

Errors: 404 — *"Order has no barcodes yet."*

12.8 GET /barcode/orders/{order_id}/skus

Filter-dropdown options for the history screen.

Response 200

```
[
  {
    "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
    "sku_code": "BOG-CLERMONT-BLK-50",
    "colour": "BLACK", "size": "50",
    "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900",
    "style_name": "CLERMONT"
  }
]
```

12.9 GET /barcode/orders/{order_id}/analytics

Planned vs generated vs balance, with an integrity proof.

Response 200

```
{
  "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899",
  "order_total": {
    "planned": 100,
    "generated": 60,
    "balance": 40,
    "active": 60,
    "retired": 0,
    "duplicates": 0,
    "half_minted": true,
    "fully_generated": false
  },
  "by_style": [
    { "style_id": "4b5c6d7e-...", "style_name": "CLERMONT", "style_code": "BOG-CLERMONT",
      "planned": 60, "minted": 60, "balance": 0 },
    { "style_id": "7e8f9001-...", "style_name": "CARNABY", "style_code": "BOG-CARNABY",
      "planned": 40, "minted": 0, "balance": 40 }
  ]
}
```

Field	Meaning
duplicates	Should always be 0 — the proof that minting stayed idempotent. Non-zero is a data-integrity incident
half_minted	Some pieces exist but not all
fully_generated	The order is complete

Alias rows are excluded from these counts. That is what the stored `is_alias` flag is for — without it, `minted` would double and `balance` would go negative.

12.10 GET /barcode/orders/{order_id}/barcodes

The paginated history table.

Query: sku_id · style_id · size · status (active | retired) · date_from · date_to · page (1) · page_size (50, max 200)

Response 200

```
{
  "order_id": "3a4b5c6d-7e8f-9001-1223-3445566778899",
  "page": 1,
  "page_size": 50,
  "total": 60,
  "pages": 2,
  "items": [
    {
      "code": "PC-004821",
      "status": "active",
      "sku_code": "BOG-CLERMONT-BLK-50",
      "style_name": "CLERMONT",
      "article": "A-4471",
      "serial": "017",
      "piece_code": "KJ2451-CLERMONT-BLK-50-017",
      "colour": "BLACK",
      "size": "50",
      "seq": 17,
      "current_stage": "PASTING",
      "generated_at": "2026-09-09T11:04:22.118000+00:00"
    }
  ]
}
```

`code` is the scannable compact id; `piece_code` is the long human identity, for reference.

MATERIALS

13. Lots and stock

Router: `app/modules/materials/router.py`

13.1 GET /materials/spec

The form definition for a material category. **Roles:** stock readers (DM, MD, HR, Cutting, Lining, Stitching, Security, Store)

Query: `category` (required) · `subtype`

Response 200

```
{
  "category": "LEATHER",
  "subtype": null,
  "required": ["thickness", "dcm"],
  "qty_field": "dcm",
  "qty_uom": "dcm",
  "filters": ["article", "colour", "thickness"]
}
```

```
{
  "category": "ACCESSORY",
  "subtype": "BUTTON",
  "required": ["size", "count"],
  "qty_field": "count",
  "qty_uom": "pcs",
  "filters": ["article", "colour", "size"]
}
```

Drive both the Add-New form and the stock search boxes off this. `article` and `colour` are always required at the top level, in addition to `required[]`.

The full table

Category / subtype	required	qty_field	qty_uom
LEATHER	thickness, dcm	dcm	dcm
LINING / PLAIN_LINING	thickness, mtrs	mtrs	mtrs
LINING / RIBS	kg	kg	kg
LINING / KNIT	pcs	pcs	pcs
ACCESSORY / BUTTON	size, count	count	pcs
ACCESSORY / ZIP	size, count	count	pcs
ACCESSORY / THREAD	thickness, mtrs	mtrs	mtrs
ACCESSORY / OTHER	description, count	count	pcs

`LINING` with no subtype defaults to `PLAIN_LINING`. **ACCESSORY always needs a subtype** — there is no generic accessory quantity.

Errors: 422 — an unknown `(category, subtype)` pair. The system does not invent a shape for a material class it was not told about.

13.2 POST /materials/lots

Create a lot. Mints a child barcode and adds stock in one transaction.

Roles: DM · MD · Cutting · Lining

Request

```
{
  "category": "LEATHER",
  "subtype": null,
  "article": "SHEEP GLASS",
  "colour": "BLACK",
  "attributes": { "thickness": "0.7", "dcm": 3400 },
  "supplier_id": "e0001111-2222-3333-4444-555566667777"
}
```

Response 201

```
{
  "lot_id": "a1338000-0000-0000-0000-0000000000338",
  "lot_barcode": "LOT-000338",
  "category": "LEATHER",
  "subtype": null,
  "article": "SHEEP GLASS",
  "colour": "BLACK",
  "on_hand": 3400.0,
  "reserved": 0.0,
  "available": 3400.0,
  "uom": "dcm"
}
```

Errors: 422 — missing a required attribute for this category, or an unknown (category, subtype) .

13.3 GET /materials/lots

★ **THE LOT PICKER for the cut screen.** Roles: stock readers.

This is the endpoint that turns "*black sheep glass, 0.7*" into a `lot_id` for `POST /production/log` .

Query: category · subtype · article · colour · thickness · size · sku_id · required

Response 200

```
{
  "count": 2,
  "required": 1035.0,
  "suggested_lot_id": "a1338000-0000-0000-0000-0000000000338",
  "options": {
    "article": ["SHEEP GLASS", "GOAT SUEDE"],
    "colour": ["BLACK", "COGNAC"],
    "thickness": ["0.7", "0.9"],
    "size": []
  },
  "lots": [
    {
      "lot_id": "a1338000-0000-0000-0000-0000000000338",
      "barcode": "LOT-000338",
      "category": "LEATHER", "subtype": null,
      "article": "SHEEP GLASS", "colour": "BLACK",
      "thickness": "0.7", "size": null,
      "uom": "dcm",
      "on_hand": 3400.0, "reserved": 0.0, "available": 3400.0,
      "last_used_for_sku": true,
      "covers_required": true
    },
    {
      "lot_id": "a1339000-0000-0000-0000-0000000000339",
      "barcode": "LOT-000339",
      "category": "LEATHER", "subtype": null,
      "article": "SHEEP GLASS", "colour": "BLACK",
      "thickness": "0.9", "size": null,
      "uom": "dcm",
      "on_hand": 400.0, "reserved": 0.0, "available": 400.0,
      "last_used_for_sku": false,
      "covers_required": false
    }
  ]
}
```

How the cut screen uses it

```
GET /materials/lots?category=LEATHER&sku_id=<sku>
  → `options` fills the cascading dropdowns
  → the row with `last_used_for_sku: true` is pre-selected – AND YOU SHOW THAT YOU DID
...narrow with &article=&colour=&thickness=
  → one row left; take its `lot_id`
POST /production/log with consumption.leather_lot_id + dcm
```

Pass required (dcm per piece × piece count) and each lot reports **covers_required** — so a lot that cannot cover the batch is greyed out **before** the cut.

Exhausted lots are returned, not hidden. A manager searching for a lot they know exists must find it, with **available: 0** explaining itself.

Three material GETs, three jobs: `/spec` returns the **form definition**; `/lots` returns the **rows with ids**; `/stock` returns **aggregate totals with the ids summed away**.

13.4 GET /materials/lots/{lot_id}

One lot, opened from the stock screen.

Response 200

```
{
  "lot_id": "a1338000-0000-0000-0000-0000000000338",
  "barcode": "LOT-000338",
  "category": "LEATHER", "subtype": null,
  "article": "SHEEP GLASS", "colour": "BLACK",
  "thickness": "0.7", "size": null,
  "uom": "dcm",
  "on_hand": 3400.0, "reserved": 1035.0, "available": 2365.0,
  "attributes": { "thickness": "0.7", "dcm": 3400 },
  "supplier_id": "e0001111-2222-3333-4444-555566667777",
  "is_active": true,
  "editable_fields": ["article", "colour", "thickness", "supplier_id"],
  "required_attributes": ["thickness", "dcm"]
}
```

`editable_fields` and `required_attributes` let the edit form render **without a second call** to `/materials/spec`.

13.5 PATCH /materials/lots/{lot_id}

Correct a lot's IDENTITY. Roles: DM · MD · Cutting · Lining

Request — send only what changes:

```
{ "thickness": "0.75", "supplier_id": "e0002222-..." }
```

Accepts: `article` · `colour` · `thickness` · `size` · `supplier_id`

category, subtype, uom and on_hand are absent on purpose. Moving LEATHER→LINING would leave a lot measured in dcm claiming to be metres, and every cut event pointing at it would silently re-denominate. Setting `on_hand` directly would make the stock and the movement history disagree with no record of who changed it — use `/adjust`.

Errors: 409 — the new spec collides with another lot. *Material is one lot per spec, or the picker shows two rows a cutter cannot tell apart.*

13.6 PATCH /materials/lots/{lot_id}/adjust

A counted stock correction — audited. Roles: DM · MD

Request

```
{ "delta": -40, "reason": "Physical count 2026-09-09: 40 dcm unaccounted after re-measure" }
```

delta is the **MOVEMENT**, not the new total. +12 adds twelve; -12 removes twelve. **reason** is 3–300 characters and **mandatory** — it is the only record of why the count and the system disagreed.

Response 200 — the full LotDetail .

Errors: 409 — would take stock below what is already reserved, or below zero.

13.7 DELETE /materials/lots/{lot_id}

RETIRE a lot. Never a hard delete. **Roles:** DM · MD

Response 200

```
{
  "lot_id": "a1338000-0000-0000-0000-0000000000338",
  "is_active": false,
  "barcode_retired": true,
  "on_hand": 0.0,
  "history_preserved": true,
  "message": "Lot retired. Cut events and consumption history are unchanged; scanning the old label now returns 410 Gone."
}
```

Present it as "retire", not "delete". Cut events point at this lot and are the consumption history the costing and traceability screens are built on.

Errors: 409 — stock is still reserved for a cut.

13.8 GET /materials/stock

Aggregate on-hand / reserved / available for a filtered material. Roles: stock readers.

Query: category · subtype · article · colour · thickness · size · required

Response 200

```
{
  "category": "LEATHER", "subtype": null,
  "article": "SHEEP GLASS", "colour": "BLACK", "thickness": "0.7", "size": null,
  "uom": "dcm",
  "on_hand": 3400.0,
  "reserved": 1035.0,
  "available": 2365.0,
  "lot_count": 1,
  "required": 3000.0,
  "short_by": 635.0,
  "suggested_supplier": {
    "supplier_id": "e0001111-2222-3333-4444-555566667777",
    "name": "S.N. TRADERS",
    "phone": "+919840012345",
    "last_rate": 1.72
  }
}
```

available is derived — **on_hand - reserved** — **never stored, so the two cannot drift.** `short_by` and `suggested_supplier` feed straight into `POST /suppliers/orders`.

13.9 `POST /materials/receive`

Record a delivery: approved and rejected separately. Roles: DM · MD

Request

```
{
  "lot_id": "a1338000-0000-0000-0000-0000000000338",
  "supplier_order_id": "50001111-2222-3333-4444-555566667777",
  "approved_qty": 2800,
  "rejected_qty": 200,
  "reserve_for_required": 1035,
  "approve_mismatch": false
}
```

Response 200

```
{
  "lot_id": "a1338000-0000-0000-0000-0000000000338",
  "on_hand": 6200.0,
  "reserved": 1035.0,
  "available": 5165.0,
  "rejected_logged": 200.0,
  "supplier_order_status": "arrived",
  "substituted": false,
  "mismatch_fields": null
}
```

- **Approved** is added to stock.
- **Rejected** is **logged** — supplier quality history.
- `reserve_for_required` holds a quantity against a known requirement.

Response 409 — a PO mismatch

```
{
  "detail": {
    "error": "po_mismatch",
    "mismatch_fields": ["colour", "thickness"],
    "message": "The delivery is COGNAC 0.9; the order was for BLACK 0.7. A DM/MD may accept this as a substitution."
  }
}
```

Re-post with `approve_mismatch: true` (DM/MD) and it is received into a **new substitute lot** — `substituted: true`, with a new `lot_id`. The original lot is not quietly re-labelled.

13.10 POST /materials/issues

Record a material issued to one garment OUTSIDE its spec. Spends stock.

Roles: DM · MD · Store Manager

The escape hatch that makes the frozen recipe acceptable. A released style's spec cannot be edited — its garments are already being issued against it — so a wrong article is corrected by recording **what was actually handed over**, not by rewriting the recipe underneath live pieces.

Request — barcodes or ids for each of the three:

```
{
  "piece_barcode": "PC-004821",
  "lot_barcode": "LOT-000112",
  "employee_barcode": "EMP-000123",
  "qty": 1,
  "note": "Zip #5 substituted for #4 - #4 out of stock"
}
```

Response 201

```
{
  "issue_id": "b0001111-2222-3333-4444-555566667777",
  "piece_id": "8f900112-2334-4556-6778-899001122334",
  "piece_code": "PC-004821",
  "material_lot_id": "a1112000-0000-0000-0000-000000000112",
  "article": "YKK ZIP #5 60CM",
  "qty": 1.0,
  "uom": "pcs",
  "source": "MANUAL",
  "available_after": 239.0,
  "note": "Zip #5 substituted for #4 - #4 out of stock",
  "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "entered_by": "Suresh Iyer"
}
```

Repeatable on purpose. Three corrections on one garment are three real events, and **none of them makes the kit checklist think a spec line was satisfied.**

Errors: 422 — neither a piece nor a lot identifier given.

14. Suppliers

Roles: DM · MD for all three.

14.1 POST /suppliers/orders

Raise a manual supplier order (ORDERED). Validates the requested article against the supplier's catalogue.

Request

```
{
  "category": "LEATHER",
  "subtype": null,
  "article": "SHEEP GLASS",
  "colour": "BLACK",
  "thickness": "0.7",
  "dcm": 3400,
  "qty": 3400,
  "supplier_id": "e0001111-2222-3333-4444-555566667777"
}
```

Omit `supplier_id` and the system suggests one from the article.

Response 201

```
{
  "order_id": "50001111-2222-3333-4444-555566667777",
  "status": "ordered",
  "article": "SHEEP GLASS",
  "qty": 3400.0,
  "uom": "dcm",
  "supplier": { "supplier_id": "e0001111-...", "name": "S.N. TRADERS",
               "phone": "+919840012345", "last_rate": 1.72 }
}
```

14.2 PATCH /suppliers/orders/{order_id}

Flip ORDERED → ARRIVED. Cues the receiving screen. **Idempotent.**

Request

```
{ "status": "arrived" }
```

Response 200

```
{ "order_id": "50001111-...", "status": "arrived", "arrived_at": "2026-09-09T08:12:00+00:00" }
```

14.3 PATCH /suppliers/orders/{order_id}/spec

Edit an ORDERED order's spec. Send only what changes.

Request

```
{ "colour": "COGNAC", "thickness": "0.9", "qty": 4000 }
```

Accepts `article` · `colour` · `thickness` · `dcm` · `qty` (must be > 0).

Response 200 — same shape as 14.2.

15. The style recipe

Prefix: `/styles/{style_id}/material-spec` · **Reads:** stock readers · **Writes:** DM · MD

What one garment of this style needs.

15.1 GET `/styles/{style_id}/material-spec`

Response 200

```
{
  "style_id": "4b5c6d7e-8f90-0112-2334-4556677889900",
  "style_name": "CLERMONT",
  "production_status": "DRAFT",
  "confirmed_at": null,
  "confirmed_by": null,
  "no_accessories": null,
  "editable": true,
  "release_blockers": ["material spec is not confirmed"],
  "lines": [
    {
      "line_id": "c0001111-2222-3333-4444-555566667777",
      "sku_id": null,
      "category": "LEATHER", "subtype": null,
      "article": "SHEEP GLASS", "colour": "BLACK", "thickness": "0.7", "size": null,
      "qty_per_piece": 34.5,
      "uom": "dcm",
      "material_lot_id": "a1338000-0000-0000-0000-0000000000338",
      "resolves_to": { "lot_id": "a1338000-...", "barcode": "LOT-000338", "available": 2365.0 },
      "resolution": "BOUND",
      "note": null,
      "is_active": true
    },
    {
      "line_id": "c0002222-3333-4444-5555-666677778888",
      "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
      "category": "ACCESSORY", "subtype": "ZIP",
      "article": "YKK ZIP #5 60CM", "colour": "BLACK", "thickness": null, "size": "60",
      "qty_per_piece": 1,
      "uom": "pcs",
      "material_lot_id": null,
      "resolves_to": null,
      "resolution": "AMBIGUOUS",
      "candidate_lot_ids": ["a1112000-...", "a1113000-..."],
      "note": "size-50 SKU only",
      "is_active": true
    }
  ]
}
```

Field	Meaning
sku_id	null = the style-wide default. Set = a per-SKU override

Field	Meaning
<code>uom</code>	Derived from <code>(category, subtype)</code> — never taken from the client
<code>resolution</code>	<code>BOUND</code> (explicit lot) · <code>RESOLVED</code> (one lot matches) · <code>NONE</code> · <code>AMBIGUOUS</code> (several — a human must pick)
<code>editable</code>	<code>false</code> once the style is <code>RELEASED</code>
<code>release_blockers</code>	Render this on the release screen — before the button, not after the rejection

15.2 PUT /styles/{style_id}/material-spec

Save the whole grid in one call. Idempotent — posting it twice is a no-op.

Request

```
{
  "lines": [
    { "category": "LEATHER", "article": "SHEEP GLASS", "colour": "BLACK",
      "thickness": "0.7", "qty_per_piece": 34.5 },
    { "category": "LINING", "subtype": "PLAIN LINING", "article": "VISCOSE",
      "colour": "BLACK", "thickness": "0.2", "qty_per_piece": 1.2 },
    { "category": "ACCESSORY", "subtype": "ZIP", "article": "YKK ZIP #5 60CM",
      "colour": "BLACK", "size": "60", "qty_per_piece": 1 },
    { "sku_id": "6d7e8f90-...", "category": "ACCESSORY", "subtype": "BUTTON",
      "article": "HORN 24L", "colour": "BROWN", "size": "24", "qty_per_piece": 4,
      "note": "size-50 SKU only" }
  ]
}
```

`uom` is accepted and **ignored** — the unit is a property of the material kind. It stays in the contract only so a client can round-trip a GET without stripping fields.

Response 200 — the full spec as in 15.1.

Errors: `409` — the style is `RELEASED`. *"Correct a released style on the floor with `POST /materials/issues`."*

15.3 POST /styles/{style_id}/material-spec/lines

Add one line. Body = one `SpecLineIn`. **Response 201** — the created line.

Errors: `409` — this style already has that material · `409` — `RELEASED`

15.4 PATCH /styles/{style_id}/material-spec/lines/{line_id}

Edit one line. Omitted fields keep their current value. **Response 200** — the updated line.

15.5 DELETE /styles/{style_id}/material-spec/lines/{line_id}

Remove a line. **SOFT** — the issue ledger still points at it, so it deactivates.

Response 200

```
{ "line_id": "c0002222-...", "is_active": false, "history_preserved": true }
```

15.6 POST /styles/{style_id}/material-spec/confirm

★ Sign the recipe off. THIS IS WHAT UNLOCKS RELEASE.

Request

```
{ "no_accessories": false }
```

Response 200

```
{
  "style_id": "4b5c6d7e-...",
  "confirmed_at": "2026-09-09T10:40:02.118000+00:00",
  "confirmed_by": "Tanveer Ahmed",
  "no_accessories": false,
  "lines": 4,
  "release_blockers": []
}
```

Deliberately a separate call from release. The DM can finish the recipe days earlier, and the release screen can render `release_blockers` **before** the button is pressed rather than explaining a rejection afterwards.

`no_accessories: true` is the DM stating "*this garment takes none*". It is **the only thing** that lets a style with an empty accessory list through the release gate — an empty list on its own is ambiguous (nobody entered them yet?), and releasing that silently is how a whole order reaches the store with no kit.

Errors: 422 — `no_accessories: true` while accessory lines exist. **A 422, not a precedence rule.** · 409 — unresolved lines remain.

15.7 POST /styles/{style_id}/material-spec/copy-from

Seed this recipe from another style's. Does NOT confirm it.

Request

```
{ "source_style_id": "7e8f9001-1223-3445-5667-788990011223",
  "include_sku_overrides": false }
```

Response 201

```
{
  "style_id": "4b5c6d7e-...",
  "copied": 4,
  "skipped": 1,
  "skipped_detail": [
    { "article": "HORN 24L", "reason": "source SKU (COGNAC · 48) has no match in this style" }
  ],
  "confirmed": false
}
```

SKU overrides copy only where both styles share a (colour, size). Unmatched ones are **reported in** `skipped_detail`, **never guessed** onto the wrong colourway — which would silently issue the wrong-colour button.

15.8 GET /styles/{style_id}/material-spec/requirement

★ `qty_ordered` × `per-piece` vs what is on the shelf.

Response 200

```
{
  "style_id": "4b5c6d7e-...",
  "style_name": "CLERMONT",
  "qty_ordered": 60,
  "badge": "short",
  "lines": [
    {
      "line_id": "c0001111-...",
      "category": "LEATHER", "article": "SHEEP GLASS", "colour": "BLACK",
      "qty_per_piece": 34.5, "uom": "dcm",
      "required": 2070.0,
      "on_hand": 3400.0, "reserved": 1035.0, "available": 2365.0,
      "short_by": 0.0,
      "status": "sufficient",
      "suggested_supplier": null
    },
    {
      "line_id": "c0002222-...",
      "category": "ACCESSORY", "subtype": "ZIP", "article": "YKK ZIP #5 60CM",
      "qty_per_piece": 1, "uom": "pcs",
      "required": 60.0,
      "on_hand": 20.0, "reserved": 0.0, "available": 20.0,
      "short_by": 40.0,
      "status": "short",
      "suggested_supplier": { "supplier_id": "e0002222-...", "name": "ZIP WORLD",
        "phone": "+912266778899", "last_rate": 1.30 }
    }
  ]
}
```

This is the screen that should stop an order, and the moment to read it is BEFORE release. Afterwards the garments exist and a shortfall is a stoppage rather than a purchase order.

`short_by` and `suggested_supplier` feed straight into `POST /suppliers/orders`.

THE FLOOR

16. Production

Router: `app/modules/production/router.py`

16.1 GET /production/operations

The configured pipeline steps. Roles: floor readers (MD, DM, HR, Supervisor, Cutting, Lining, Stitching)

Response 200


```
[
  { "id": "0p000001-0000-0000-0000-000000000001", "code": "LEATHER_CUTTING", "label": "Leather cutting", "sequence": 10 },
  { "id": "0p000002-0000-0000-0000-000000000002", "code": "LINING_CUTTING", "label": "Lining cutting", "sequence": 15 },
  { "id": "0p000003-0000-0000-0000-000000000003", "code": "FUSING", "label": "Fusing", "sequence": 20 },
  { "id": "0p000004-0000-0000-0000-000000000004", "code": "PASTING", "label": "Pasting", "sequence": 30 },
  { "id": "0p000005-0000-0000-0000-000000000005", "code": "LINE_STITCHING", "label": "Line stitching", "sequence": 40 },
  { "id": "0p000006-0000-0000-0000-000000000006", "code": "SHELL_STITCHING", "label": "Shell stitching", "sequence": 50 },
  { "id": "0p000007-0000-0000-0000-000000000007", "code": "FINAL_FINISH", "label": "Final finish", "sequence": 60 },
  { "id": "0p000008-0000-0000-0000-000000000008", "code": "FINAL_INSPECTION", "label": "Final inspection", "sequence": 70 },
  { "id": "0p000009-0000-0000-0000-000000000009", "code": "PACKAGE_EXPORT", "label": "Package & export", "sequence": 80 }
]
```

label and rates are config; the ORDER is code. Do not order client-side by `sequence` and assume it matches the chain — the canonical order lives in the enum.

16.2 GET /production/skus

The friendly SKU picker. Roles: any authenticated (client-scoped).

Query: `order_id · style_id`

Response 200

```
[
  {
    "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
    "code": "BOG-CLERMONT-BLK-50",
    "label": "CLERMONT · BLACK · 50",
    "order_number": "BOG-SS27-001",
    "style_name": "CLERMONT",
    "color_code": "BLK",
    "color_name": "BLACK",
    "size": "50",
    "qty_ordered": 20
  }
]
```

Show `label`, never the UUID.

16.3 GET /production/piece-state

★ **THE SCAN-TIME READ. One piece, and whether it can be logged right now.**

Roles: floor readers.

Query: `code` or `piece_id` (one required) · `employee_barcode` or `employee_id` (optional but recommended)

For a fully automatic scan, send `code` + `employee_barcode`.

Response 200 — ready

```
{
  "piece": {
    "piece_id": "8f900112-2334-4556-6778-899001122334",
    "code": "PC-004821",
    "piece_code": "KJ2451-CLERMONT-BLK-50-017",
    "serial": "017", "seq": 17,
    "article": "A-4471", "style_name": "CLERMONT",
    "colour": "BLACK", "size": "50",
    "order_number": "BOG-SS27-001",
    "needs_lining": true
  },
  "drawer": { "drawer_id": "90011223-...", "code": "DRW-0042", "state": "sended",
    "holding": "HOLDING BOTH", "leather_in": true, "lining_in": true },
  "completed_stages": ["LEATHER_CUTTING", "LINING_CUTTING", "FUSING", "PASTING"],
  "current_stage": "PASTING",
  "current_stage_label": "Pasting",
  "next_stage": "LINE_STITCHING",
  "next_stage_requires_consumption": false,
  "suggested_dcm_per_piece": null,
  "display_stage": "LINE_STITCHING",
  "display_label": "Line stitching",
  "in_store": false,
  "material_requirement": {
    "kit_required": true, "kit_status": "ISSUED", "outstanding": []
  },
  "stages": [
    { "stage": "LEATHER_CUTTING", "state": "completed", "requires_consumption": true },
    { "stage": "LINING_CUTTING", "state": "completed", "requires_consumption": true },
    { "stage": "FUSING", "state": "completed" },
    { "stage": "PASTING", "state": "completed" },
    { "stage": "LINE_STITCHING", "state": "next" },
    { "stage": "SHELL_STITCHING", "state": "locked", "gate": "sequence",
      "reason": "line stitching is not complete" },
    { "stage": "FINAL_FINISH", "state": "locked", "gate": "sequence",
      "reason": "shell stitching is not complete" },
    { "stage": "FINAL_INSPECTION", "state": "locked", "gate": "sequence",
      "reason": "final finish is not complete" },
    { "stage": "PACKAGE_EXPORT", "state": "locked", "gate": "sequence",
      "reason": "final inspection is not complete" }
  ],
  "sku": { "sku_id": "6d7e8f90-...", "total": 20, "done": 12, "remaining": 8, "closed": false },
  "can_log_next": true,
  "actor": {
    "employee_id": "d4e5f607-...", "name": "Farid Ansari",
    "designation": "LINE_TAILOR", "present_today": true, "skill_ok": true
  },
  "blockers": [],
  "ready_to_log": true
}
```

Response 200 — blocked

```
{
  "next_stage": "LINE_STITCHING",
  "drawer": { "code": "DRW-0042", "state": "received", "holding": "HOLDING BOTH" },
  "can_log_next": true,
  "actor": { "name": "Farid Ansari", "designation": "LINE_TAILOR",
    "present_today": true, "skill_ok": true },
  "blockers": [
    { "gate": "merge", "reason": "the drawer has not been sent – ask the store to send DRW-0042" }
  ],
  "ready_to_log": false
}
```

Response 200 — no worker scanned

```
{ "next_stage": "LINE_STITCHING", "actor": null, "blockers": [], "ready_to_log": null }
```

The three states of ready_to_log

Value	Meaning	UI
true	POST the log now	Enable Log
false	Show blockers	Disable Log, list the reasons
null	No employee sent — the question is unanswered	"Scan the worker's card." NOT "blocked"

`blockers[].gate` ∈ employee · attendance · role · sequence · merge · consumption · completed

`stages[].state` ∈ completed · next · locked (with `gate` + `reason`) · not_applicable (a lining cut on an unlined piece)

This read uses the very same predicates `POST /log enforces` — so a card the UI opens is a card the log will accept.

Two extras: `suggested_dcm_per_piece` prefills the cut screen (a **suggestion, never a substitution**), and `sku.closed: true` means **stop offering this style for scanning**.

Errors: 422 — neither `code` nor `piece_id` · 404 — unknown code · 410 — retired

16.4 POST /production/log

★ THE WHOLE FLOOR'S LOGGING SURFACE

Roles (door gate): MD · DM · HR · Supervisor · Cutting · Lining · Stitching **STORE_MANAGER is deliberately excluded** — store functions only.

Behind the door gate sits the per-stage **Gate 1**, and passing the door does **not** mean the stage will log.

Gate 1 — the complete role → stage map

Stage	Role that may log it
LEATHER_CUTTING	cutting_manager
LINING_CUTTING	lining_manager
FUSING · PASTING · LINE_STITCHING · SHELL_STITCHING · FINAL_FINISH	stitching_manager
FINAL_INSPECTION	— no manager role —

Stage	Role that may log it
PACKAGE_EXPORT	— no manager role —

MD, DM and HR bypass Gate 1 entirely and log any stage.

⚠ **The last two stages have no manager role, and this is the 403 that looks like a bug.** A `stitching_manager` who scans a garment at final inspection gets a 403 reading "*The final inspection stage is logged by its own manager (direct_manager, managing_director)*" — the message falls back to naming DM/MD precisely **because the allow-list is empty**. The stitching manager's authority ends at `FINAL_FINISH`. Build the screen so a non-DM/MD/HR login does not offer those two stages, rather than letting the operator scan into a refusal.

There is a per-operation escape hatch. `operation_access` is a real table and Gate 1 consults it after the static map says no, so an MD can grant one role one operation without a deploy. A 403 is therefore not provably permanent — but it is what you get by default.

Gate 2 — the complete designation → stage map

A designation outside its stage's set returns a `skill_warnings[]` entry and **still logs the piece**. Never render these as errors.

Stage	Ordinary designations
LEATHER_CUTTING	CUTTER
LINING_CUTTING	CUTTER · LINING_CUTTER
FUSING	CUTTER · FUSER
PASTING	PASTER · TAILOR
LINE_STITCHING	LINE_TAILOR · TAILOR
SHELL_STITCHING	SHELL_TAILOR · TAILOR
FINAL_FINISH	FINISHER · TAILOR
FINAL_INSPECTION	FINISHER · INSPECTOR
PACKAGE_EXPORT	FINISHER · PACKER

`HELPER` and `SUPERVISOR` **never warn** at any stage, and an **unknown designation fails open** — so an HR backfill gap never blocks the floor.

Request — the barcode door (cut screen)

```
{
  "actor": { "employee_barcode": "EMP-000123" },
  "targets": { "piece_barcodes": ["PC-004821", "PC-004822", "PC-004823"] },
  "work_date": "2026-09-09",
  "consumption": {
    "article": "SHEEP GLASS",
    "colour": "BLACK",
    "thickness": "0.7",
    "dcm": 34.5
  }
}
```

Request — the manual door (pipeline)

```
{
  "actor": { "employee_id": "d4e5f607-1829-3a4b-5c6d-7e8f9001a2b3" },
  "targets": { "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
    "piece_seqs": [17, 18, 19] },
  "work_date": "2026-09-09"
}
```

Request — a dry run

```
{ "actor": {...}, "targets": {...}, "work_date": "2026-09-09", "preview": true }
```

Field	Notes
actor	employee_barcode or employee_id . Both may be sent; they must agree
targets	piece_barcodes or (sku_id + piece_seqs)
work_date	Required
screen_context	Optional and normally ignored — derived from the role. Only DM/MD/HR may override, and they must for the two cut stages
consumption	Cut stages only — see below
preview	Dry run: compute every bucket, write nothing

There is no stage field, and there never will be.

consumption — three doors

```
{ "leather_lot_id": "a1338000-...", "dcm": 34.5 }
```

```
{ "article": "SHEEP GLASS", "colour": "BLACK", "thickness": "0.7", "dcm": 34.5 }
```

```
{ "article": "SHEEP GLASS", "colour": "BLACK", "use_style_spec": true }
```

thickness is **optional and free text** — it was a required dropdown, which meant a hide whose thickness was not already in the list could not be logged at all, and the floor's answer to that is to pick a wrong value.

use_style_spec is **OFF by default**. Omitting both dcm and the flag still returns a 422, so no existing client silently changes behaviour on the one branch that guards the leather ledger.

Response 201

```

{
  "stage": "LINE_STITCHING",
  "stages": ["LINE_STITCHING"],
  "stage_by_piece": { "PC-004821": "LINE_STITCHING", "PC-004822": "LINE_STITCHING" },
  "count_logged": 2,
  "logged": ["PC-004821", "PC-004822"],
  "rework": [],
  "not_found": [],
  "completed": [],
  "sequence_blocked": [],
  "skill_blocked": [],
  "merge_blocked": ["PC-004823"],
  "role_blocked": [],
  "blocked": [
    { "piece": "PC-004823", "stage": "LINE_STITCHING", "gate": "merge",
      "reason": "drawer DRW-0044 has not been sent" }
  ],
  "skill_warnings": [
    { "piece": "PC-004821", "employee": "Farid Ansari", "designation": "LINE_TAILOR",
      "stage": "LINE_STITCHING", "note": "" }
  ],
  "message": "2 pieces logged at LINE_STITCHING. 1 blocked - its drawer has not been sent.",
  "screen_role_warning": null,
  "stock_warning": null,
  "consumption_recorded": null,
  "consumption_source": null,
  "preview": false,
  "drawer_by_piece": {
    "PC-004821": { "drawer_id": "90011223-...", "code": "DRW-0042",
      "state": "sended", "holding": "HOLDING BOTH",
      "leather_in": true, "lining_in": true }
  },
  "sku_progress": { "sku_id": "6d7e8f90-...", "stage": "LINE_STITCHING",
    "total": 20, "done": 14, "remaining": 6, "closed": false },
  "kit_by_piece": {
    "PC-004821": { "kit_required": true, "kit_status": "ISSUED", "outstanding": [] }
  }
}

```

Response 201 — a cut, with consumption

```

{
  "stage": "LEATHER_CUTTING",
  "count_logged": 29,
  "logged": ["PC-004821", "..."],
  "consumption_recorded": {
    "leather_lot_id": "a1338000-...",
    "article": "SHEEP GLASS", "colour": "BLACK",
    "dcm_per_piece": 34.5, "pieces": 29,
    "total_consumed": 1000.5,
    "on_hand_after": 2399.5, "uom": "dcm"
  },
  "consumption_source": "typed",
  "stock_warning": null,
  "message": "29 pieces cut. 1,000.5 dcm consumed from lot LOT-000338."
}

```

Response 201 — a cut that went short

```
{
  "stage": "LEATHER_CUTTING",
  "count_logged": 29,
  "stock_warning": {
    "lot_id": "a1339000-...", "article": "SHEEP GLASS", "colour": "BLACK",
    "uom": "dcm",
    "requested": 1000.5, "available_before": 400.0, "short_by": 600.5,
    "on_hand_after": 0.0,
    "note": "The cut IS recorded – the garments are physically cut. Stock has drifted; adjust the lot after a count."
  }
}
```

Response field guide

Field	Meaning
stage	"MIXED" when a PIPELINE batch spanned several stages — read <code>stage_by_piece</code> . <code>null</code> only when nothing resolved
logged / rework / not_found / completed	Classic buckets. <code>completed</code> = past the final stage
sequence_blocked / skill_blocked / merge_blocked / role_blocked	Per-piece refusals
blocked[]	Every rejection with its cause. <code>gate ∈ role · sequence · merge · not_cut · completed</code>
skill_warnings[]	Gate 2 anomalies that STILL LOGGED. Never render as errors
role_blocked	Pieces whose inferred stage this role may not log, in a mixed batch where other stages were permitted . An all-denied batch is still a 403
screen_role_warning	A leather cutter scanned on the lining screen. A warning, not a block
sku_progress	closed: true → stop offering this style for scanning
kit_by_piece	Deliberately lean — a 40-piece batch carrying full requirement blocks would dwarf the response
consumption_source	"typed" or "style_spec"

Errors

Code	When
403	Gate 1 — this role may not log this stage. The message names the roles that can. Also: an all-denied mixed batch
403	Door gate — the role may not reach the log at all
404	No piece number in <code>piece_seqs</code> exists for this SKU · no lot matches the typed spec
409	Several lots match the spec. The message names up to five candidates with their availability
422	A cut with neither <code>dcm</code> nor <code>use_style_spec</code>
422	Pieces in the batch disagree on the spec dcm — the message names both values
422	An invalid <code>screen_context</code>
422	The employee is not present today

The 409 you must build for

```
{
  "detail": "3 LEATHER lots match that spec – say which one. Add a thickness, or send the lot id
directly. Candidates: SHEEP GLASS · BLACK · 0.7 (2365.0 dcm available, lot a1338000-...); SHEEP GLASS
· BLACK · 0.9 (400.0 dcm available, lot a1339000-...); ..."
}
```

Render the candidates and let the operator pick. Silently taking the first would decrement stock from a lot the manager never chose.

16.5 GET /production/events

The raw event feed. Roles: floor readers only — **not CLIENT or VIEWER.** It names the employee who worked each piece.

Query: sku_id · employee_id · start · end

Response 200

```
[
  {
    "id": "e0001111-2222-3333-4444-555566667777",
    "sku_id": "6d7e8f90-...",
    "operation_id": "0p000001-...",
    "employee_id": "c3d4e5f6-...",
    "work_date": "2026-09-09",
    "qty": 1,
    "entered_by": "Suresh Iyer",
    "piece_id": "8f900112-..."
  }
]
```

qty is always 1 — one event, one piece.

16.6 GET /production/skus/{sku_id}/pieces

Every piece of one SKU with its stage and eligibility. Query: operation_id

Response 200

```
{
  "sku_id": "6d7e8f90-0112-2334-4556-677889900112",
  "sku_code": "BOG-CLERMONT-BLK-50",
  "order_id": "3a4b5c6d-...",
  "colour": "BLACK", "size": "50",
  "operation_id": "0p000005-...", "operation_code": "LINE_STITCHING",
  "total": 20, "done": 14, "pending": 6,
  "pieces": [
    { "piece_id": "8f900112-...", "code": "KJ2451-CLERMONT-BLK-50-017", "seq": 17,
      "current_stage": "LINE_STITCHING", "current_stage_label": "Line stitching",
      "done_at_op": true },
    { "piece_id": "90011223-...", "code": "KJ2451-CLERMONT-BLK-50-018", "seq": 18,
      "current_stage": "PASTING", "current_stage_label": "Pasting",
      "done_at_op": false }
  ]
}
```

seq is what the manual door takes in piece_seqs .

16.7 GET /production/styles/{style_id}/progress

Per-stage completed counts — the live progress card.

Response 200

```
{
  "style_id": "4b5c6d7e-...",
  "style_name": "CLERMONT",
  "qty_ordered": 60,
  "stages": {
    "LEATHER_CUTTING": 60, "LINING_CUTTING": 60,
    "FUSING": 58, "PASTING": 55,
    "LINE_STITCHING": 40, "SHELL_STITCHING": 22,
    "FINAL_FINISH": 10, "FINAL_INSPECTION": 4, "PACKAGE_EXPORT": 0
  }
}
```

16.8 Removed routes

Route	Status	Message
POST /production/cutting	410	"Pieces mint at breakdown upload; log cutting via POST /production/log with screen_context=LEATHER_CUT."
POST /production/scan	410	"...replaced by POST /production/log (two-door)."

410, not 404, so a stale frontend fails loudly with a message naming the replacement.

17. ~~Drawers~~ — WITHDRAWN

This entire chapter is retired. Every `/drawers/*` route returns 404. The store is now a state on the garment: see [§0.5](#) for `/store/*`, and [§0.2](#) for the endpoint-by-endpoint replacement table.

The text below is kept for one release so the old behaviour stays readable while the drawer tables are still in the database (retained, unwritten, for audit). It describes endpoints that no longer exist — do not build against it.

Retired reference (historical)

Router: `app/modules/drawers/router.py`

`_FLOOR` = MD · DM · Cutting · Lining · Stitching · Supervisor · **Store Manager** `_SENDERS` = MD · DM · **Store Manager** · **Stitching Manager**

17.1 GET /drawers

The Drawers List. Roles: `_FLOOR`

Query: `state` · `code` (case-insensitive contains) · `seq_from` / `seq_to` · `has_piece` · `sendable` · `sort` (`recent` default | `seq`) · `pin_codes` (repeatable, max 20) · `limit` (500, max 2000) · `offset`

Response 200

```

{
  "total": 200,
  "count": 3,
  "items": [
    {
      "drawer_id": "90011223-3445-5667-7889-900112233445",
      "seq": 42,
      "code": "DRW-0042",
      "state": "received",
      "holding": "HOLDING BOTH",
      "leather_in": true, "lining_in": true, "accessories_in": true,
      "kit_required": true,
      "needs_lining": true,
      "lining_reason": null,
      "complete": true,
      "piece_id": "8f900112-...",
      "piece_code": "KJ2451-CLERMONT-BLK-50-017",
      "piece_serial": "017",
      "can_send": true,
      "barcode_id": "b0011223-...",
      "barcode": "DRW-0042",
      "caption": "Drawer 42",
      "barcode_status": "active",
      "last_activity_at": "2026-09-09T11:58:14.220000+00:00",
      "last_activity": "received",
      "pinned": true
    },
    {
      "drawer_id": "a0112233-...", "seq": 117, "code": "DRW-0117",
      "state": "holding_leather", "holding": "HOLDING LEATHER",
      "leather_in": true, "lining_in": false, "accessories_in": false,
      "kit_required": true,
      "needs_lining": true,
      "lining_reason": "its style name contains 'KNIT'",
      "complete": false,
      "piece_code": "KJ2451-CLERMONT-BLK-50-023", "piece_serial": "023",
      "can_send": false,
      "barcode": "DRW-0117", "barcode_status": "active",
      "last_activity_at": "2026-09-09T11:44:02.881000+00:00",
      "last_activity": "scanned",
      "pinned": false
    },
    {
      "drawer_id": "b0223344-...", "seq": 8, "code": "DRW-0008",
      "state": "waiting", "holding": "EMPTY",
      "leather_in": false, "lining_in": false, "accessories_in": false,
      "kit_required": false, "needs_lining": true, "complete": false,
      "piece_id": null, "piece_code": null,
      "can_send": false,
      "barcode": null, "barcode_status": null,
      "last_activity_at": "2026-09-09T10:02:00.000000+00:00",
      "last_activity": "released",
      "pinned": false
    }
  ]
}

```

Field guide

Field	Meaning
state	The lifecycle position
holding	What is physically inside. A different question — they stop agreeing at <code>received</code> and <code>sended</code>
needs_lining	The EFFECTIVE requirement , resolved from the style/SKU/cut history — not the stored flag, which is written once at release and wrong for most of a live order
lining_reason	Why, when the effective verdict disagrees with the flag
can_send	Drives the Send checkbox. Applies the same rule the send endpoint enforces
last_activity	<code>merged scanned received sent released</code> . Render with <code>last_activity_at</code> : <code>"sent · 2 min ago"</code>
pinned	Floated by <code>pin_codes</code> . Band these separately
barcode: null	A broken label — cannot be scanned. Show it; print nothing for it

`sendable=true` is the send queue. `sort=seq` is the print sheet.

Errors: `422` — an invalid `state` (the message lists the seven), or `seq_from > seq_to` .

17.2 GET /drawers/{drawer_id} and GET /drawers/by-code/{code}

One drawer, opened from the list or found by the search box. Same body.

Response `200`

```
{
  "drawer_id": "90011223-3445-5667-7889-900112233445",
  "code": "DRW-0042", "seq": 42,
  "state": "received",
  "holding": "HOLDING BOTH",
  "leather_in": true, "lining_in": true, "accessories_in": true,
  "kit_required": true,
  "needs_lining": true, "lining_reason": null,
  "awaiting": [],
  "complete": true,
  "received_at": "2026-09-09T11:58:14.220000+00:00",
  "sended_at": null,
  "sent": false,
  "can_send": true,
  "piece": {
    "piece_id": "8f900112-...", "code": "PC-004821",
    "piece_code": "KJ2451-CLERMONT-BLK-50-017", "serial": "017",
    "article": "A-4471", "style_name": "CLERMONT",
    "colour": "BLACK", "size": "50", "order_number": "BOG-SS27-001"
  }
}
```

Errors: `404` — *"No drawer with code 'DRW-9999'."*

17.3 POST /drawers/store-scan

★ **Record a part arriving in its drawer. Roles:** `_FLOOR`

The order is employee → drawer → piece, and all three are enforced here.

Request — a cut part

```
{
  "employee_barcode": "EMP-000123",
  "drawer_barcode": "DRW-0042",
  "piece_barcode": "PC-004821"
}
```

`part` omitted → **the server infers LEATHER vs LINING** from the piece's cut history and what the drawer is missing.

Request — the accessory kit

```
{
  "employee_barcode": "EMP-000123",
  "drawer_barcode": "DRW-0042",
  "piece_barcode": "PC-004821",
  "part": "ACCESSORY"
}
```

Request — a partial or substituted kit

```
{
  "employee_barcode": "EMP-000123",
  "drawer_barcode": "DRW-0042",
  "piece_barcode": "PC-004821",
  "part": "ACCESSORY",
  "lines": [
    { "spec_id": "c0002222-...", "qty": 1, "material_lot_id": "a1113000-..." }
  ]
}
```

Response 200

```

{
  "drawer_code": "DRW-0042",
  "piece_code": "PC-004821",
  "state": "received",
  "needs_lining": true,
  "lining_reason": null,
  "awaiting": [],
  "ready_for_received": true,
  "part": "LINING",
  "part_inferred": true,
  "employee_id": "c3d4e5f6-...",
  "holding": "HOLDING BOTH",
  "auto_received": true,
  "sent": false,
  "next_action": "This drawer is complete. Select it in the Drawers List and send it to release the garment into line stitching.",
  "late_kit": false,
  "kit": {
    "status": "PARTIAL",
    "summary_line": "2 of 3 accessory lines issued",
    "issued_now": [],
    "already_issued": [
      { "spec_id": "c0002222-...", "category": "ACCESSORY", "subtype": "ZIP",
        "article": "YKK ZIP #5 60CM", "colour": "BLACK", "size": "60",
        "qty_per_piece": 1, "qty": 1, "uom": "pcs", "issued": 1,
        "lot_id": "a1112000-...", "available_after": 239.0 }
    ],
    "outstanding": [
      { "spec_id": "c0003333-...", "article": "HORN 24L", "colour": "BROWN",
        "qty": 4, "uom": "pcs" }
    ],
    "unresolved": [
      { "spec_id": "c0004444-...", "article": "THREAD 40/2", "colour": "BLACK",
        "reason": "AMBIGUOUS",
        "candidate_lot_ids": ["a1120000-...", "a1121000-..."],
        "note": "two lots carry this article - pick one" }
    ],
    "stock_warnings": [],
    "complete": false
  }
}

```

Field guide

Field	Meaning
<code>part_inferred</code>	true = the server chose . Show which part it picked
<code>auto_received</code>	Completeness advanced the drawer to RECEIVED . Worth a visible confirmation
<code>sent: false</code> + <code>next_action</code>	Scanning is NOT completion . Say so explicitly
<code>late_kit</code>	The kit was issued into an already-RECEIVED drawer. Normal only where the spec arrived after its garments
<code>kit.status</code>	<code>NOT_REQUIRED</code> → hide the checklist , not render an empty one
<code>kit.unresolved</code>	<code>reason</code> ∈ <code>NONE</code> (no lot carries this article) <code>AMBIGUOUS</code> (several do)
<code>kit.stock_warnings</code>	The issue was still recorded ; the line went short

part=ACCESSORY spends stock and is never inferred. The scan is **idempotent** — a second tap issues nothing and returns the same 200.

The **kit** block comes back on **EVERY** scan, cut parts included: the person holding the leather is the one who also has to find the buttons.

Errors

Code	When
422	No employee identifier — <i>"Scan the employee barcode first"</i>
422	No drawer or no piece identifier
404	The employee is not active — <i>"...so work cannot be recorded against them"</i>
409	The piece belongs to a different drawer. The merge map is the authority
410	A retired card

17.4 POST /drawers/send

★ Send one or many drawers — and the pieces in them — onward.

Roles: _SENDERS (MD, DM, Store Manager, **Stitching Manager**)

Request

```
{ "drawer_ids": ["90011223-...", "a0112233-...", "b0223344-..."] }
```

There is no destination to pick. Lining is upstream of the store, so a merged drawer has exactly one way forward.

Response 200

```
{
  "requested": 3,
  "count_sent": 2,
  "sent": [
    { "drawer_id": "90011223-...", "code": "DRW-0042", "piece_code": "PC-004821" },
    { "drawer_id": "b0223344-...", "code": "DRW-0055", "piece_code": "PC-004830" }
  ],
  "not_ready": [
    { "drawer_id": "a0112233-...", "code": "DRW-0117",
      "reason": "still HOLDING LEATHER – the lining has not been scanned in" }
  ],
  "not_found": [],
  "pieces_released": ["PC-004821", "PC-004830"],
  "message": "2 of 3 drawers sent. 2 garments released into line stitching."
}
```

Check **count_sent**, **not the HTTP status**. Partial accept: one drawer that is not ready never loses the twenty that are.

pieces_released are exactly the pieces that just became eligible for LINE_STITCHING.

17.5 GET /drawers/pool

Pool size, free drawers, and the waiting list. Roles: _FLOOR

Response 200

```
{
  "pool_size": 200,
  "free": 12,
  "in_use": 188,
  "pieces_waiting_for_drawer": 3,
  "shortfall": 3
}
```

pieces_waiting_for_drawer are minted pieces with barcodes but **no drawer** — they cannot be stored, so they cannot pass the merge gate. **shortfall** is what a DM would have to add right now to clear the list.

17.6 POST /drawers/pool

Add N permanent barcoded drawers, then drain the waiting list into them.

Roles: DM · MD only. **One-way — the pool never shrinks.**

Request

```
{ "add": 50 }
```

`add` is 1–1000. **The cap is a typo guard**, not a technical limit: `add: 20000` would mint twenty thousand permanent drawers and there is no un-mint.

Response 201

```
{
  "added": 50,
  "codes": ["DRW-0201", "DRW-0202", "...", "DRW-0250"],
  "allocated": 3,
  "pieces_merged": ["PC-004890", "PC-004891", "PC-004892"],
  "status": { "pool_size": 250, "free": 47, "in_use": 203,
    "pieces_waiting_for_drawer": 0, "shortfall": 0 },
  "by": "Tanveer Ahmed"
}
```

Print the returned codes. A drawer with no printed label cannot be scanned, so it is a drawer that does not exist as far as the floor is concerned.

Growing **also drains the waiting list** in the same call — growing the pool while leaving pieces waiting would leave a DM staring at empty drawers beside a waiting list.

17.7 POST /drawers/allocate-waiting

Merge drawer-less pieces into whatever drawers are currently free. Roles: `_SENDERS`

The waiting list drains itself as garments ship (PACKAGE_EXPORT recycles a drawer to WAITING), but nothing merges the next waiting piece into it automatically. This is that step — **safe to call repeatedly and a no-op when there is nothing to place.**

Response 200

```
{
  "allocated": 3,
  "pieces_merged": ["PC-004890", "PC-004891", "PC-004892"],
  "status": { "pool_size": 200, "free": 9, "in_use": 191,
    "pieces_waiting_for_drawer": 0, "shortfall": 0 }
}
```

17.8 POST /drawers/{drawer_id}/receive — DEPRECATED

Kept working for one release so a frontend mid-deploy does not break.

RECEIVED is now reached automatically the moment a drawer holds everything its garment needs, and **SENDED** is done in bulk through `POST /drawers/send`. **Move to those two.**

PEOPLE

18. Attendance

Router: `app/modules/attendance/router.py`

Every WRITE requires an operator login: SECURITY · HR · MD · DM. Both doors share one dependency, so the barcode door and the manual door can never drift apart.

Location tracking is removed. No route asks for or stores a position. `lat` / `lon` are **accepted and ignored** so a client that still sends them gets a 201, not a 422.

18.1 POST /attendance/scan-check-in

★ **THE PRIMARY DOOR** — the operator scans a worker's card. **Roles:** operators.

Request

```
{ "employee_barcode": "EMP-000123", "direction": "in", "proxy": false, "reason": null }
```

Field	Notes
<code>employee_barcode</code>	Required
<code>direction</code>	"in" or "out"
<code>proxy</code>	<code>true</code> marks the manual fallback (card would not scan)
<code>reason</code>	Free-text context. No longer mandatory — with no geofence there is nothing to excuse
<code>lat</code> / <code>lon</code>	Accepted, ignored

Response 200


```
{
  "id": "f0001111-2222-3333-4444-555566667777",
  "employee_id": "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",
  "name": "Ramesh Kumar",
  "work_date": "2026-09-09",
  "check_in_at": "2026-09-09T03:32:10.000000+00:00",
  "check_out_at": null,
  "source": "self",
  "is_late": false,
  "is_short": false,
  "is_overtime": false,
  "recorded_by_user_id": "a1b2c3d4-...",
  "idempotent_replay": false
}
```

Check-in is idempotent — a re-tap returns the existing row with `idempotent_replay: true` and changes nothing.

Errors: 404 unknown card · 410 retired card · 403 not an operator

18.2 POST /attendance/check-in · POST /attendance/check-out

An operator records their OWN arrival / departure. Roles: operators.

Request: the body is **optional** — `{}`, `{"lat":..., "lon":...}` (ignored), or no body at all. Identity comes from the token; the timestamp is set **server-side** to block client-side clock manipulation.

Response 201 / 200 — an `AttendanceRead` as in 18.1.

Errors: 422 — the login has no linked `employee_id`.

18.3 POST /attendance/proxy/check-in · POST /attendance/proxy/check-out

The manual door — the operator types employees in. Roles: operators. Any wage type.

Request

```
{ "employee_ids": ["c3d4e5f6-...", "d4e5f607-..."] }
```

Response 201 / 200 — a **list** of `AttendanceRead`, each with `source: "proxy"` and `recorded_by_user_id` set to the operator.

18.4 POST /attendance/daily-workers

Onboard a daily-wage worker on the floor. Roles: DM · HR · MD — **deliberately NOT widened to Supervisor**. Adding people to the payroll is a deliberate authority.

Request

```
{ "name": "Kamal Das", "designation": "HELPER", "phone": null, "daily_rate": 550 }
```

`phone` is optional — the worker gets **no login**, so there is nothing to authenticate with it.

Response 201

```
{ "id": "e5f60718-293a-4b5c-6d7e-8f9001a2b3c4",
  "name": "Kamal Das",
  "wage_type": "piece_rate" }
```

18.5 GET /attendance/today

The whole-floor roster. Roles: operators + floor leads (Supervisor, Cutting, Lining, Stitching).

Response 200 — a list of AttendanceRead for today.

18.6 GET /attendance/history

One employee's history. Roles: operators + floor leads.

Query: start (required) · end (required) · employee_id (required)

Errors: 403 not a reader · 422 employee_id omitted

18.7 GET /attendance/me

The caller's own history. Roles: any authenticated.

Query: start · end — defaults to the last 30 days.

Returns [] when the login has no linked employee_id.

18.8 ~~ GET /attendance/me/status ~~ — REMOVED 2026-09-19

The route is unrouted. It 404s. There is no replacement, and no screen should be built for it.

It served a live shift countdown — *how much of my shift is left* — for a login looking at its own row.

Why it went. Only an operator (SECURITY · HR · MD · DM) holds a login at all, and an operator is at the gate scanning other people's cards, not watching a timer for themselves. The people a countdown would actually suit — the shop floor — have no login to see it with.

GET /attendance/me (§18.7) still serves a staff login's own record. It is a history, not a countdown.

18.9 ~~ GET /attendance/config · PATCH /attendance/config ~~ — REMOVED 2026-09-19

Both routes are unrouted. They 404s. Shift policy is configured once, server-side, and there is no API for it.

Why the read went. Nothing on the floor sets policy per session. It is configured once and the backend applies it on every punch.

Why the write went with it. An edit form that cannot load its own current values is a screen that overwrites policy with whatever the frontend happened to hold. Leaving the write door open without the read door was the worse half to keep. If policy needs changing, both come back together, or it is a DB/seed change.

THE POLICY ITSELF IS UNAFFECTED. AttendanceService still reads ShiftConfig on every check-in and check-out to set is_late / is_short / is_overtime, and the wage run still prices against it. What is gone is the API that read and wrote the row, not the row.

The geofence fields (factory_lat , factory_lon , radius_m) still exist on the model and remain unused.

18.10 PATCH /attendance/{attendance_id}

Correct a punch — and re-allocate the day's work with it. Roles: SECURITY · HR · MD · DM (every attendance operator).

The mistake this exists for is not "nobody was here". Security scans a card at the gate; the card said MAJID, the worker who actually did the cutting was SALIM. By the time anybody notices, MAJID has a day's events against his name — **and the wage, because wages are computed from the events, not from the attendance.**

So changing `employee_id` is not editing a field. It is "*this was the wrong person*", and the events filed under that name are just as wrong as the punch. **They move in the same action**, or the money follows the wrong worker no matter how many times the attendance is corrected.

Request — every field optional

```
{
  "employee_id": "e5f60718-293a-4b5c-6d7e-8f9001a2b3c4",
  "reason": "card swapped at the gate"
}
```

Field	Effect
<code>employee_id</code>	re-allocation — that day's production events move too
<code>check_in_at</code>	corrected arrival; the flags are recomputed
<code>check_out_at</code>	corrected departure; the flags are recomputed
<code>reason</code>	free text, written to the <code>ATTENDANCE_CORRECTED</code> audit row

Response 200

```
{
  "attendance_id": "a1b2c3d4-5e6f-7081-92a3-b4c5d6e7f809",
  "employee_id": "e5f60718-293a-4b5c-6d7e-8f9001a2b3c4",
  "work_date": "2026-09-18",
  "check_in_at": "2026-09-18T03:32:10Z",
  "check_out_at": null,
  "is_late": false,
  "is_short": false,
  "is_overtime": false,
  "production_events_moved": 12,
  "message": "Re-allocated to the correct worker; 12 production event(s) for 2026-09-18 moved with it."
}
```

`production_events_moved` is **0** for a pure time correction and non-zero only for a re-allocation. Show it — it is the size of what the operator just did.

The flags are recomputed, never carried forward. A corrected punch still claiming `is_short` from the old times would be worse than not correcting it at all.

Only that day moves. A swapped card is one day's mistake; yesterday's events stay where they are.

Errors

Co de	When
404	no such attendance record, or the employee to re-allocate to does not exist
409	the target is already marked present that day. One punch per worker per day is a database constraint, so it is refused in words rather than surfacing as a 500: " <i>PADMA is already marked present on 2026-09-18...</i> "
409	the day falls inside a CLOSED payroll run — that run is the document the cash was counted against and is never recomputed, so the fix is a payroll adjustment, not a quiet edit underneath it

18.11 DELETE /attendance/{attendance_id}?reason=

Remove a punch that should never have been made. Roles: HR · MD · DM **only**.

SECURITY is deliberately absent: an operator who mis-scans should be able to **correct** it without going to find anybody, but **removing** the record decides whether somebody is paid for the day at all, and that stays with HR and management.

reason is required and the API **422s** when it is blank or whitespace. It is the only thing that makes the deletion reviewable afterwards.

It refuses when the worker has production events that day — 409, naming the better operation:

```
{
  "detail": "This worker has 3 production event(s) on 2026-09-18, so the work was really done – the usual cause is a swapped card, not an absent worker. Re-allocate the attendance to the right person instead (PATCH this record with the correct employee_id); the events move with it. If the work genuinely should not exist, delete those events first."
}
```

Deleting would leave those events dated to a day the worker was never marked present — a state **the production log's own presence gate would have refused to create**.

Response 200

```
{
  "attendance_id": "a1b2c3d4-5e6f-7081-92a3-b4c5d6e7f809",
  "deleted": true,
  "message": "Attendance record removed."
}
```

Errors: 404 not found · 422 no reason · 409 production events that day · 409 closed payroll run · 403 SECURITY.

Design note. One screen, two buttons, very different weights. **Re-allocate** is primary with a worker picker and a live count on the confirm ("Move 12 events to SALIM"). **Delete** sits behind a second click with a mandatory reason box — and when the 409 comes back, offer *"Re-allocate instead"* as the action, because that is what the operator actually meant.

MONEY

19. Wages

Router: app/modules/wages/router.py · **Roles:** DM · MD · HR for reads; **DM · MD only** for recompute, reopen, close and delete.

Wages are not behind bare get_current_user anywhere. CLIENT is a real role with a real login, and **per-piece labour rates are the factory's cost structure**.

Route order is load-bearing — every static segment is declared before /runs/{run_id} .

19.1 GET /wages/orders

The payroll landing screen. Query: on (coverage date, default today) · unpriced_only

Response 200

```
[
  {
    "order_number": "BOG-SS27-001",
    "client_name": "BOGGI MILANO",
    "styles": 2,
    "styles_priced": 1,
    "delivery_deadline": "2026-11-30"
  }
]
```

`styles_priced` / `styles` is the card's badge. Click through to `/wages/styles?order_number=...`.

19.2 GET /wages/styles

Query: `order_number` · `client_id` · `unpriced_only` · `on`

Response 200

```
[
  {
    "style_code": "BOG-CLERMONT",
    "style_name": "CLERMONT",
    "order_number": "BOG-SS27-001",
    "client_name": "BOGGI MILANO",
    "rated_operations": 6,
    "total_operations": 9,
    "qty_ordered": 60
  }
]
```

`rated_operations` / `total_operations` is why this endpoint exists. Unpriced styles pay **zero, silently**, unless someone looks.

Style codes, never style ids.

19.3 GET /wages/rate-sheet

Query: `style_code` (required) · `on` (default today)

Response 200

```
{
  "style_code": "BOG-CLERMONT",
  "style_name": "CLERMONT",
  "on": "2026-09-09",
  "rows": [
    { "operation_code": "LEATHER_CUTTING", "operation_label": "Leather cutting",
      "sequence": 10, "rate": 12.50, "effective_from": "2026-08-01" },
    { "operation_code": "LINING_CUTTING", "operation_label": "Lining cutting",
      "sequence": 15, "rate": 6.00, "effective_from": "2026-08-01" },
    { "operation_code": "FINAL_INSPECTION", "operation_label": "Final inspection",
      "sequence": 70, "rate": null, "effective_from": null }
  ]
}
```

`rate: null` means unpriced — it will pay zero. Highlight it.

19.4 GET /wages/rate-history

Query: `style_code` · `operation_code` (both required)

Response 200

```
{
  "style_code": "BOG-CLERMONT",
  "operation_code": "LEATHER_CUTTING",
  "rows": [
    { "rate": 12.50, "effective_from": "2026-08-01", "set_at": "2026-07-28T09:00:00+00:00" },
    { "rate": 11.75, "effective_from": "2026-04-01", "set_at": "2026-03-27T14:12:00+00:00" }
  ]
}
```

Newest first. **A mid-period rate change prices each day at the rate in force that day.**

19.5 POST /wages/rates · POST /wages/rates/bulk

Single-cell and whole-sheet saves.

Single

```
{ "style_code": "BOG-CLERMONT", "operation_code": "LEATHER_CUTTING",
  "rate": 13.00, "effective_from": "2026-10-01" }
```

Bulk — all codes resolved first, then written **in one transaction**:

```
{
  "style_code": "BOG-CLERMONT",
  "effective_from": "2026-10-01",
  "lines": [
    { "operation_code": "LEATHER_CUTTING", "rate": 13.00 },
    { "operation_code": "LINING_CUTTING", "rate": 6.50 },
    { "operation_code": "FINAL_INSPECTION", "rate": 3.00 }
  ]
}
```

Response 200

```
{ "style_code": "BOG-CLERMONT", "effective_from": "2026-10-01", "rates_written": 3 }
```

Prefer bulk when saving a whole sheet.

19.6 POST /wages/runs

★ **COMPUTE ONE PAYROLL.** Side-effecting and non-idempotent.

Request — a piece run

```
{
  "run_kind": "piece",
  "style_code": "BOG-CLERMONT",
  "period_start": "2026-09-01",
  "period_end": "2026-09-15",
  "freeze": true
}
```

Request — a monthly run

```
{
  "run_kind": "monthly",
  "period_start": "2026-09-01",
  "period_end": "2026-09-30",
  "order_number": "BOG-SS27-001",
  "freeze": false
}
```

Field	Notes
run_kind	piece (default) — pays PIECE_RATE workers. style_code or order_number is REQUIRED. monthly — pays salaried staff from the calendar; no scope required. combined cannot be created
order_number / style_code	Not both. On a piece run they define what is paid; on a monthly run they are a label
freeze	true → CLOSED immediately. false → an OPEN draft you can recompute freely

Response 201

```
{
  "id": "70001111-2222-3333-4444-555566667777",
  "period_start": "2026-09-01",
  "period_end": "2026-09-15",
  "status": "closed",
  "run_kind": "piece",
  "scope_order_number": null,
  "scope_style_code": "BOG-CLERMONT",
  "scope_is_label": false,
  "piece_rate_only": true,
  "monthly_only": false,
  "total_amount": 48250.00,
  "total_pieces": 3860,
  "employee_count": 22,
  "unrated_operations": [
    { "operation_code": "FINAL_INSPECTION", "operation_label": "Final inspection",
      "style_code": "BOG-CLERMONT", "pieces": 40 }
  ],
  "gap_days": 0,
  "recomputed": false,
  "recompute_count": 0,
  "reopen_count": 0,
  "computed_at": "2026-09-16T05:02:11.004000+00:00",
  "lines": [
    { "employee_id": "c3d4e5f6-...", "employee_name": "Ramesh Kumar",
      "designation": "CUTTER", "wage_type": "piece_rate",
      "pieces": 240, "amount": 3000.00,
      "detail": [
        { "style_code": "BOG-CLERMONT", "operation_code": "LEATHER_CUTTING",
          "pieces": 240, "rate": 12.50, "amount": 3000.00 }
      ]
    }
  ]
}
```

Read before paying anyone

Field	Why
unrated_operations	These stages paid zero. Non-empty means someone worked for nothing

Field	Why
<code>gap_days</code>	Days in the window no run covers
<code>scope_is_label</code>	true = the order/style is DECORATION. Do not print " <i>CLERMONT payroll</i> " over a sheet that paid every salaried person
<code>piece_rate_only</code> / <code>monthly_only</code>	Say so, rather than let a manager read a one-style run as a full payroll

Errors

Code	When
422	A piece run with no scope. " <i>Two dates alone would pay every piece of every order in that window.</i> "
422	Both <code>order_number</code> and <code>style_code</code>
422	<code>run_kind: "combined"</code>
409	Overlap. The message names the blocking run and what to do about it

A piece run and a monthly run over the same window do NOT conflict. Their populations are disjoint. Compute both.

19.7 GET /wages/runs

Run history, newest computed first.

Query: `run_kind` · `order_number` · `style_code` · `date_from` · `date_to` · `status` · `limit` (50) · `offset`

Dates **overlap-match**, so a fortnight straddling the month boundary still comes back.

Response 200 — a list of `WageRunSummary` (as in 19.6, with `lines: []`).

The recompute workflow this exists for: filter by `style_code` (or `order_number`) plus a date range, read the id off the matching row, then POST recompute.

19.8 GET /wages/runs/{run_id}

Re-read a frozen run — payslip detail, no recomputation.

Response 200 — a `WageRunDetail`: everything in the summary, plus `last_recomputed_at` / `_by`, `last_reopened_at` / `_by`, `last_reopen_reason`, and full `lines`.

19.9 POST /wages/runs/{run_id}/close

FREEZE a draft. Roles: DM · MD

Idempotent — closing an already-closed run returns it unchanged, because that is what a double-click is.

Errors: 409 — **refuses to freeze a run with no lines.** *Locking a window in which nobody is paid is never what was meant.*

19.10 POST /wages/runs/{run_id}/recompute

Discard the lines and rebuild them from current events and rates, for the **same window and scope. Roles:** DM · MD — HR is deliberately excluded.

Request (*optional*)


```
{ "confirm_closed": false }
```

Free on an OPEN run. On a CLOSED run this **409s** and tells you to reopen first.

`confirm_closed: true` is the one-call escape hatch. It works, it stamps the recompute, and it **records no reason** — which is why the reopen door exists.

Response 200 — the rebuilt `WageRunDetail`. The run **keeps its id, window and scope**; `recompute_count` increments and the actor is stamped, **so a payslip reprinted afterwards is identifiably a different document from the one paid against.**

19.11 POST /wages/runs/{run_id}/reopen

★ **UNFREEZE a CLOSED run. Roles:** DM · MD only.

Request

```
{ "reason": "Cutting rate for Sept corrected from 12.50 to 13.00 after the client re-quote." }
```

The reason is mandatory and stored, and belongs on every reissued payslip for that period.

Response 200

```
{
  "id": "70001111-...",
  "status": "open",
  "reopen_count": 1,
  "last_reopened_at": "2026-09-20T07:11:03.220000+00:00",
  "last_reopened_by": "Tanveer Ahmed",
  "last_reopen_reason": "Cutting rate for Sept corrected..."
}
```

Deliberately a separate call from recompute. A manager who must press reopen, type why, then press recompute cannot rewrite a paid payslip by mistyping a run id.

19.12 DELETE /wages/runs/{run_id}

Delete a run and its lines outright. Roles: DM · MD

Request (*optional*)

```
{ "confirm_closed": true }
```

Response 200

```
{ "run_id": "70001111-...", "deleted": true, "lines_deleted": 22, "was_closed": false }
```

The escape hatch the 409 has always named. `create_run` commits an OPEN run **before** any line is written, so a compute that died halfway left a committed, empty run permanently occupying that window and blocking every later run over those dates.

Refuses a CLOSED run unless `confirm_closed: true`. A frozen run is the document the cash was counted against. **To CHANGE a frozen run, reopen and recompute** — that keeps the audit trail. Deleting is for runs that should never have existed.

19.13 GET /wages/runs/{run_id}/breakdown

★ One frozen run folded three ways.

Response 200

```
{
  "run_id": "70001111-...",
  "total_amount": 48250.00,
  "total_pieces": 3860,
  "by_style": [
    {
      "style_code": "BOG-CLERMONT", "style_name": "CLERMONT",
      "pieces": 3860, "amount": 48250.00,
      "stages": [
        { "operation_code": "LEATHER-CUTTING", "operation_label": "Leather cutting",
          "sequence": 10, "pieces": 600, "amount": 7500.00,
          "employees": [
            { "employee_id": "c3d4e5f6-...", "name": "Ramesh Kumar",
              "pieces": 240, "amount": 3000.00 }
          ]
        }
      ]
    }
  ],
  "by_stage": [
    { "operation_code": "LEATHER-CUTTING", "operation_label": "Leather cutting",
      "sequence": 10, "pieces": 600, "amount": 7500.00 }
  ],
  "by_employee": [
    { "employee_id": "c3d4e5f6-...", "name": "Ramesh Kumar",
      "designation": "CUTTER", "pieces": 240, "amount": 3000.00 }
  ]
}
```

All three folds come off the same frozen row set, so the three tabs cannot disagree with each other or with the run total — **which they will if the frontend sums them itself.**

19.14 GET /wages/runs/{run_id}/pieces

★ Per-piece payroll detail. Query: `style_code · limit (500) · offset`

Response 200

```
{
  "run_id": "70001111-...",
  "total": 3860,
  "count": 2,
  "items": [
    {
      "piece_code": "PC-004821",
      "piece_serial": "017",
      "style_code": "BOG-CLERMONT",
      "colour": "BLACK", "size": "50",
      "operation_code": "LEATHER_CUTTING",
      "work_date": "2026-09-04",
      "employee_id": "c3d4e5f6-...",
      "employee_name": "Ramesh Kumar",
      "employee_barcode": "EMP-000123",
      "rate": 12.50,
      "amount": 12.50,
      "note": null
    },
    {
      "piece_code": "PC-004821",
      "operation_code": "FINAL_INSPECTION",
      "work_date": "2026-09-14",
      "employee_name": "Priya Nair",
      "employee_barcode": "EMP-000201",
      "rate": null,
      "amount": null,
      "note": "the stage was unrated at run time"
    }
  ]
}
```

Amounts come from the run's FROZEN rate for that (employee, style, stage) cell. Nothing is re-priced, so a rate corrected after the run closed does not change what this says the run paid.

A row with amount: null carries a note explaining why — a monthly worker, or an unrated stage. **Deliberately not a zero, which would read as work worth nothing.**

19.15 GET /wages/ledger

Every computed run, latest computed first.

Query: order_number · style_code · date_from · date_to · status · limit (10) · offset

Response 200

```
{
  "total": 34,
  "count": 1,
  "items": [
    {
      "run_id": "70001111-...",
      "run_kind": "piece",
      "period_start": "2026-09-01", "period_end": "2026-09-15",
      "status": "closed",
      "scope_order_number": null, "scope_style_code": "BOG-CLERMONT",
      "scope_is_label": false,
      "total_amount": 48250.00, "total_pieces": 3860, "employee_count": 22,
      "computed_at": "2026-09-16T05:02:11.004000+00:00",
      "recompute_count": 1,
      "reopen_count": 1
    }
  ]
}
```

Ordered by when a run was computed, not by its period — the question the ledger answers is *"what did we just run"*. `recompute_count` and `reopen_count` make a rebuilt run **visibly different** from one that has not been touched.

Reads the frozen rows only. Nothing here is re-derived.

READS

20. Analytics

Router: `app/modules/analytics/router.py` · **Roles:** any authenticated, **client-scoped**.

A CLIENT login is pinned to its own `client_id`. **A cross-tenant id resolves to 404** — existence itself is information.

20.1 GET /analytics/explorer

The left-panel nav tree. Query: `include_pieces` (default `true`)

Response `200`

```
{
  "clients": [
    {
      "client_id": "7c9e6679-...", "name": "BOGGI MILANO",
      "orders": [
        {
          "order_id": "3a4b5c6d-...", "order_number": "BOG-SS27-001",
          "styles": [
            {
              "style_id": "4b5c6d7e-...", "name": "CLERMONT", "qty_ordered": 60,
              "skus": [
                { "sku_id": "6d7e8f90-...", "code": "BOG-CLERMONT-BLK-50",
                  "colour": "BLACK", "size": "50", "qty_ordered": 20,
                  "pieces": 20 }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

20.2 GET /analytics/orders/{order_id}/tree

Order landing view — styles with piece counts and stage distribution.

Response 200

```
{
  "order_id": "3a4b5c6d-...",
  "order_number": "BOG-SS27-001",
  "client_name": "BOGGI MILANO",
  "styles": [
    {
      "style_id": "4b5c6d7e-...", "name": "CLERMONT",
      "qty_ordered": 60, "pieces": 60,
      "stage_distribution": {
        "LEATHER_CUTTING": 0, "FUSING": 2, "PASTING": 15,
        "LINE_STITCHING": 18, "SHELL_STITCHING": 12,
        "FINAL_FINISH": 6, "FINAL_INSPECTION": 4, "PACKAGE_EXPORT": 3
      }
    }
  ]
}
```

stage_distribution counts where pieces are *now*, not how many have passed.

20.3 GET /analytics/styles/{style_id}/detail

Every piece of one style with its full stage history — employee, date and time per event.

20.4 GET /analytics/pieces/detail

One piece by **piece_code** OR (**sku_code** + **seq**). Header plus grouped stages.

20.5 GET /analytics/pieces/{piece_code}/story

★ THE PIECE LIFE STORY.

Response 200

```
{
  "piece_code": "PC-004821",
  "long_code": "KJ2451-CLERMONT-BLK-50-017",
  "serial": "017",
  "article": "A-4471",
  "style_name": "CLERMONT", "colour": "BLACK", "size": "50",
  "order_number": "BOG-SS27-001", "client_name": "BOGGI MILANO",
  "needs_lining": true,
  "current_stage": "SHELL_STITCHING",
  "waiting_on": null,
  "drawer": { "code": "DRW-0042", "state": "waiting" },
  "story": [
    { "stage": "LEATHER_CUTTING", "label": "Leather cutting",
      "employee_name": "Ramesh Kumar", "employee_barcode": "EMP-000123",
      "work_date": "2026-09-04", "at": "2026-09-04T05:12:00+00:00",
      "rework": false,
      "consumption": { "lot_barcode": "LOT-000338", "article": "SHEEP GLASS",
        "colour": "BLACK", "qty": 34.5, "uom": "dcm" } },
    { "stage": "LINING_CUTTING", "employee_name": "Anita Roy",
      "work_date": "2026-09-04", "rework": false,
      "consumption": { "lot_barcode": "LOT-000512", "article": "VISCOSE",
        "qty": 1.2, "uom": "mtrs" } },
    { "stage": "FUSING", "employee_name": "Farid Ansari",
      "work_date": "2026-09-06", "rework": false, "consumption": null },
    { "stage": "PASTING", "employee_name": "Farid Ansari",
      "work_date": "2026-09-07", "rework": true, "consumption": null }
  ]
}
```

rework: true marks a piece logged at a stage it had already passed — legitimate, flagged, never blocked.

waiting_on names what is holding the piece when it cannot advance.

20.6 GET /analytics/consumption

Leather consumed per style, summed from cut events. Query: `order_id` · `style_id`

Response 200

```
{
  "rows": [
    {
      "style_id": "4b5c6d7e-...", "style_name": "CLERMONT",
      "article": "SHEEP GLASS", "colour": "BLACK",
      "pieces_cut": 60,
      "consumed": 2070.0, "uom": "dcm",
      "avg_per_piece": 34.5,
      "on_hand": 1330.0, "available": 295.0
    }
  ]
}
```

20.7 GET /analytics/employee-rates

Per-employee rate / piece / earnings analytics. Roles: DM · MD · **HR only** — this is cost data.

Query: start (required) · end (required) · employee_id · style_code

20.8 Removed endpoints — 410 Gone

Endpoint	Replacement
GET /analytics/overview	"Factory-wide figures live on the role dashboards: /dashboard/direct-manager , or /dashboard/cutting /lining /stitching /store ."
GET /analytics/alerts/stage-spread	"Bottleneck and alerts are now on GET /dashboard/alerts , which every manager role may read."
GET /analytics/alerts/freight-risk	Same

410, not deleted, so a stale frontend gets a message naming the replacement instead of "the server is broken".

21. Dashboards

Router: app/modules/dashboard/router.py

Roles: MD · DM · HR · Supervisor · Cutting · Lining · Stitching · **Store Manager. CLIENT and VIEWER are not admitted** — these expose worker names and floor data.

Every dashboard is READ-ONLY. No bodies, no writes.

21.1 GET /dashboard/alerts

★ **THE ALERTS, FOR EVERY MANAGER. Query:** today (override for freight risk)

Response 200

```

{
  "bottleneck": {
    "stage": "PASTING",
    "stage_label": "Pasting",
    "queued": 41,
    "throughput_per_day": 12,
    "backlog_days": 3.4
  },
  "stage_spread": [
    { "stage": "LINE_STITCHING", "cut": 100, "reached": 58, "gap": 42 },
    { "stage": "SHELL_STITCHING", "cut": 100, "reached": 34, "gap": 66 }
  ],
  "freight_risk": [
    {
      "order_id": "3a4b5c6d-...", "order_number": "BOG-SS27-001",
      "client_name": "BOGGI MILANO",
      "sea_cutoff_date": "2026-10-05",
      "days_to_cutoff": 6,
      "qty_ordered": 100, "qty_completed": 3,
      "risk": "high"
    }
  ],
  "alert_count": 3
}

```

Block	Meaning
<code>bottleneck</code>	The deepest queue — the stage with the most work waiting. Not "the first unfinished stage", which is wherever the line happens to have got to and is not actionable
<code>stage_spread</code>	Where each downstream stage lags the leather cut. The gap is true WIP in flight
<code>freight_risk</code>	Orders approaching the sea cut-off. Missing it means air freight — the single biggest margin event in the business

`alert_count` is what a badge should render. Empty lists are a good day, not a missing feature.

21.2 The four stage dashboards

Endpoint	Query	Contains
<code>GET /dashboard/cutting</code>	<code>order_id</code>	~11 grouped queries: production KPIs, current order, per-cutter performance, leather lots, per-order progress, 14-day trend
<code>GET /dashboard/lining</code>	<code>order_id</code>	~12: the same for the lining leg, plus upcoming work — leather-cut pieces not yet lining-cut
<code>GET /dashboard/stitching</code>	<code>order_id</code>	~10: top KPIs, the pre-store (fusing/pasting) and post-store (line/shell/final) blocks, the store handoff, the running style's funnel, per-stage employee performance
<code>GET /dashboard/store</code>	<code>style_id · state · material_type</code>	~4: drawer KPIs, current styles in store, the filterable drawer grid, held drawers, empty drawers

Illustrative — `GET /dashboard/cutting`


```
{
  "kpis": { "pieces_cut_today": 84, "pieces_cut_7d": 512,
    "active_cutters": 6, "avg_per_cutter_per_day": 14.2,
    "leather_consumed_7d": 17664.0, "uom": "dcm" },
  "current_order": { "order_id": "3a4b5c6d-...", "order_number": "BOG-SS27-001",
    "client_name": "BOGGI MILANO",
    "qty_ordered": 100, "cut": 60, "remaining": 40 },
  "employees": [
    { "employee_id": "c3d4e5f6-...", "name": "Ramesh Kumar", "designation": "CUTTER",
      "pieces_7d": 168, "pieces_today": 24, "avg_dcm_per_piece": 34.6 }
  ],
  "lots": [
    { "lot_id": "a1338000-...", "barcode": "LOT-000338", "article": "SHEEP GLASS",
      "colour": "BLACK", "on_hand": 1330.0, "reserved": 1035.0, "available": 295.0,
      "uom": "dcm" }
  ],
  "orders": [
    { "order_number": "BOG-SS27-001", "qty_ordered": 100, "cut": 60, "pct": 60.0 }
  ],
  "trend": [
    { "date": "2026-09-03", "pieces": 72 },
    { "date": "2026-09-04", "pieces": 88 }
  ]
}
```

21.3 The consumption grids

Endpoint	Default stage
GET /dashboard/cutting/consumption	LEATHER_CUTTING
GET /dashboard/lining/consumption	LINING_CUTTING

Query: order_id · employee_id · start · end · stage · include_unmeasured

Response 200

```
[
  {
    "piece_code": "PC-004821",
    "serial": "017",
    "style_name": "CLERMONT", "colour": "BLACK", "size": "50",
    "stage": "LEATHER_CUTTING",
    "work_date": "2026-09-04",
    "employee_name": "Ramesh Kumar",
    "lot_barcode": "LOT-000338",
    "article": "SHEEP GLASS",
    "actual_consumption": 34.5,
    "uom": "dcm",
    "expected": null, "variance": null, "waste": null
  }
]
```

ONE STAGE PER RESPONSE. This previously returned **both** cut stages while joining the leather lot, so lining-cut events appeared with a blank lot and their quantities counted toward leather totals.

stage must be LEATHER_CUTTING or LINING_CUTTING — anything else is a **422**, not a silently empty list. Only the cut stages record consumption; asking about any other is a question with no answer.

include_unmeasured — lining consumption is optional, so a lining cut logged without a quantity is excluded by default. Pass **true** to see those events with a null quantity **rather than not at all**.

expected / **variance** / **waste** are **null** — they need a BOM, which is Phase 2.

21.4 Employee drill-downs

GET /dashboard/cutting/employees/{id} · /lining/employees/{id} · /stitching/employees/{id}

Response 200

```
[
  {
    "piece_code": "PC-004821", "serial": "017",
    "style_name": "CLERMONT", "colour": "BLACK", "size": "50",
    "stage": "LEATHER_CUTTING", "current_stage": "SHELL_STITCHING",
    "work_date": "2026-09-04",
    "last_worked_at": "2026-09-04T05:12:00+00:00" }
]
```

21.5 Piece tracking — one handler, six URLs

```
GET /dashboard/pieces/{piece_code}          ← canonical
GET /dashboard/cutting/pieces/{piece_code}
GET /dashboard/lining/pieces/{piece_code}
GET /dashboard/store/pieces/{piece_code}
GET /dashboard/stitching/pieces/{piece_code}
GET /dashboard/direct-manager/pieces/{piece_code}
```

All six return the identical body. A piece's history is the same history whichever screen is asking; six implementations would drift the moment one gained a field.

Response 200

```
{
  "piece_code": "PC-004821",
  "long_code": "KJ2451-CLERMONT-BLK-50-017",
  "serial": "017",
  "style_name": "CLERMONT", "colour": "BLACK", "size": "50",
  "order_number": "BOG-SS27-001",
  "current_stage": "SHELL_STITCHING",
  "display_stage": "SHELL_STITCHING",
  "drawer": { "code": "DRW-0042", "state": "waiting", "holding": "EMPTY" },
  "history": [
    { "stage": "LEATHER_CUTTING", "label": "Leather cutting", "sequence": 10,
      "employee_name": "Ramesh Kumar", "work_date": "2026-09-04",
      "at": "2026-09-04T05:12:00+00:00", "rework": false,
      "consumption": { "lot_barcode": "LOT-000338", "article": "SHEEP GLASS",
        "qty": 34.5, "uom": "dcm" } },
    { "stage": "STORE", "label": "In store", "sequence": 35,
      "employee_name": null, "work_date": "2026-09-08",
      "at": "2026-09-08T11:58:14+00:00", "rework": false,
      "consumption": null, "derived": true }
  ]
}
```

The **STORE** row is a **derived overlay**, not a production event — **derived: true**.

Errors: 404 — "No piece with code '...'. Scan the barcode again, or check whether the label is a drawer or employee card rather than a piece."

21.6 Store drill-downs

Endpoint	Returns
GET /dashboard/store/drawers/{id}	Full drawer info + who cut the leather and lining it holds
GET /dashboard/store/drawers/{id}/movement	The drawer's movement history from the audit trail
GET /dashboard/store/traceability	Who cut what. Query: piece_code · style_id · material_type

GET /dashboard/store/traceability

```
[
  {
    "piece_code": "PC-004821", "serial": "017",
    "style_name": "CLERMONT", "colour": "BLACK", "size": "50",
    "material_type": "LEATHER",
    "cutter_name": "Ramesh Kumar", "cutter_barcode": "EMP-000123",
    "work_date": "2026-09-04",
    "lot_barcode": "LOT-000338", "article": "SHEEP GLASS",
    "qty": 34.5, "uom": "dcm",
    "drawer_code": "DRW-0042", "drawer_state": "waiting"
  }
]
```

21.7 The Direct Manager dashboard

GET /dashboard/direct-manager — the factory-wide control panel, **composed from the four stage dashboards' own aggregates**, so this screen and the four can never disagree.

~15 grouped queries: overall production, department performance, the stage pipeline and bottleneck, production rate, quality, attendance, store send/receive, per-order progress and the 14-day trend.

Read meta.unsupported. Several quality and costing figures are **null by design** because no table backs them yet. Render them as "not tracked", never as zero.

```
{
  "...": "...",
  "meta": {
    "unsupported": ["defect_rate", "rework_cost", "material_cost_per_piece",
                  "on_time_delivery_pct"]
  }
}
```

Two drill-downs:

- GET /dashboard/direct-manager/orders/{order_id} — an order's complete journey across every stage, **with the stage currently holding the largest backlog**. 404 if unknown.
- GET /dashboard/direct-manager/styles/{style_id} — per-stage quantities for a style, end to end. 404 if unknown.

APPENDICES

40. The mock data pack

Every ID below is used consistently across every example in this document. Drop these in and **one order flows end to end**.

40.1 The scenario

BOGGI MILANO orders 100 leather garments for SS27 across two styles. **CLERMONT** (60 units, lined) is mid-production. **CARNABY** (40 units) is released but not started. Garment **PC-004821** — a size-50 black CLERMONT, serial 017 — is the thread running through every example.

40.2 Identity constants

```
export const IDS = {
  // logins
  user_md: "a1b2c3d4-e5f6-0718-293a-4b5c6d7e8f90", // Tanveer Ahmed
  user_cutting: "b2c3d4e5-f607-1829-3a4b-5c6d7e8f9001", // Suresh Iyer
  user_store: "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2",

  // employees (NO logins)
  emp_cutter: "c3d4e5f6-0718-293a-4b5c-6d7e8f9001a2", // Ramesh Kumar, CUTTER
  emp_tailor: "d4e5f607-1829-3a4b-5c6d-7e8f9001a2b3", // Farid Ansari, LINE_TAILOR
  emp_helper: "e5f60718-293a-4b5c-6d7e-8f9001a2b3c4", // Kamal Das, HELPER
  bc_cutter: "EMP-000123",
  bc_tailor: "EMP-000124",

  // commercial
  client: "7c9e6679-7425-40de-944b-e07fc1f90ae7", // BOGGI MILANO
  order: "3a4b5c6d-7e8f-9001-1223-3445566778899", // BOG-SS27-001
  style_clermont: "4b5c6d7e-8f90-0112-2334-4556677889900",
  style_carnaby: "7e8f9001-1223-3445-5667-788990011223",
  sku_blk_50: "6d7e8f90-0112-2334-4556-677889900112", // BOG-CLERMONT-BLK-50

  // the piece
  piece: "8f900112-2334-4556-6778-899001122334",
  piece_code: "PC-004821",
  piece_long: "KJ2451-CLERMONT-BLK-50-017",

  // store
  drawer: "90011223-3445-5667-7889-900112233445", // DRW-0042
  drawer_partial: "a0112233-4455-6677-8899-001122334455", // DRW-0117
  drawer_empty: "b0223344-5566-7788-9900-112233445566", // DRW-0008

  // materials
  lot_leather: "a1338000-0000-0000-0000-0000000000338", // LOT-000338
  lot_leather_2: "a1339000-0000-0000-0000-0000000000339", // LOT-000339
  lot_lining: "a1512000-0000-0000-0000-0000000000512", // LOT-000512
  lot_zip: "a1112000-0000-0000-0000-0000000000112", // LOT-000112
  supplier: "e0001111-2222-3333-4444-555566667777", // S.N. TRADERS
  supplier_zip: "e0002222-3333-4444-5555-666677778888", // ZIP WORLD
  supplier_order: "50001111-2222-3333-4444-555566667777",

  // recipe
  spec_leather: "c0001111-2222-3333-4444-555566667777",
  spec_zip: "c0002222-3333-4444-5555-666677778888",
  spec_button: "c0003333-4444-5555-6666-777788889999",

  // operations
  op_leather_cut: "0p000001-0000-0000-0000-000000000001",
  op_lining_cut: "0p000002-0000-0000-0000-000000000002",
  op_fusing: "0p000003-0000-0000-0000-000000000003",
  op_pasting: "0p000004-0000-0000-0000-000000000004",
  op_line_stitch: "0p000005-0000-0000-0000-000000000005",

  // wages
  wage_run: "70001111-2222-3333-4444-555566667777",
};
```

40.3 The consistent numbers

The order — BOG-SS27-001, 100 units, delivery 2026-11-30, **sea cut-off 2026-10-05**.

Style	Article	Units	Lined	Status	Minted
CLERMONT	A-4471	60	✓	RELEASED	60
CARNABY	A-4482	40	✓	RELEASED	40

CLERMONT SKUs — BLACK 46:10 · 48:20 · **50:20** · COGNAC 48:10

The recipe (per garment)

Category	Article	Qty	UOM
LEATHER	SHEEP GLASS BLACK 0.7	34.5	dcm
LINING	VISCOSE BLACK 0.2	1.2	mtrs
ACCESSORY / ZIP	YKK ZIP #5 60CM	1	pcs
ACCESSORY / BUTTON	HORN 24L (<i>size-50 SKU only</i>)	4	pcs

Stock

Lot	Article	On hand	Reserved	Available	UOM
LOT-000338	SHEEP GLASS BLACK 0.7	3400 → 1330	1035	295	dcm
LOT-000339	SHEEP GLASS BLACK 0.9	400	0	400	dcm
LOT-000512	VISCOSE BLACK	2000	0	2000	mtrs
LOT-000112	YKK ZIP #5 60CM	240	0	240	pcs

PC-004821 's journey — cut 09-04 (Ramesh, LOT-000338, 34.5 dcm) → lining cut 09-04 (Anita, LOT-000512, 1.2 m) → fused 09-06 → pasted 09-07 (*reworked*) → stored 09-08 in DRW-0042 → sent → line-stitched 09-09 (Farid) → currently at SHELL_STITCHING.

The wage run — piece run, BOG-CLERMONT, 2026-09-01 → 09-15, **closed**, ₹48,250 · 3,860 pieces · 22 employees.

FINAL_INSPECTION **unrated**.

40.4 A minimal MSW handler set

```
import { http, HttpResponse } from "msw";
import { IDS } from "../ids";
const V1 = "/api/v1";

export const handlers = [
  http.post(`${V1}/auth/login`, () =>
    HttpResponse.json({ access_token: "mock.jwt", token_type: "bearer",
      role: "cutting_manager", name: "Suresh Iyer",
      user_id: IDS.user_cutting, must_change_password: false })),

  http.get(`${V1}/barcode/resolve`, ({ request }) => {
    const code = new URL(request.url).searchParams.get("code");
    if (code === IDS.bc_cutter) return HttpResponse.json(RESOLVE_EMPLOYEE);
    if (code === IDS.piece_code) return HttpResponse.json(RESOLVE_PIECE);
    if (code === "DRW-0042") return HttpResponse.json(RESOLVE_DRAWER);
    if (code === "EMP-000999")
      return HttpResponse.json({ detail: "This barcode was deactivated..." },
        { status: 410 });
    return HttpResponse.json({ detail: `Unknown barcode '${code}'`,
      { status: 404 });
  }),

  http.get(`${V1}/production/piece-state`, () =>
    HttpResponse.json(PIECE_STATE_READY)), // §16.3

  http.post(`${V1}/production/log`, async ({ request }) => {
    const body = await request.json();
    if (body.preview) return HttpResponse.json({ ...LOG_RESULT, preview: true },
      { status: 201 });
    return HttpResponse.json(LOG_RESULT, { status: 201 });
  }),

  http.get(`${V1}/drawers`, () => HttpResponse.json(DRAWERS_PAGE)), // §17.1
  http.post(`${V1}/drawers/store-scan`, () => HttpResponse.json(STORE_SCAN)),
  http.post(`${V1}/drawers/send`, () => HttpResponse.json(SEND_RESULT)),
  http.get(`${V1}/drawers/pool`, () =>
    HttpResponse.json({ pool_size: 200, free: 12, in_use: 188,
      pieces_waiting_for_drawer: 3, shortfall: 3 })),

  http.get(`${V1}/materials/lots`, () => HttpResponse.json(LOT_PICKER)), // §13.3

  http.get(`${V1}/imports/breakdown/:order`, () =>
    HttpResponse.json(BREAKDOWN_TABLE)), // §11.4
  http.post(`${V1}/imports/breakdown/:order/release`, () =>
    HttpResponse.json(RELEASE_RESULT, { status: 201 })), // §11.9

  http.post(`${V1}/wages/runs`, () =>
    HttpResponse.json(WAGE_RUN, { status: 201 })), // §19.6
  http.get(`${V1}/dashboard/alerts`, () => HttpResponse.json(ALERTS)), // §21.1
];
```

40.5 Every error state worth building for

```
export const ERRORS = {
  // barcode
  unknownCode: [404, { detail: "Unknown barcode 'PC-999999'." }],
  retiredCode: [410, { detail: "This barcode was deactivated. The underlying " +
    "record and history are intact; issue a new label " +
    "to scan again." }],

  // production
  roleBlocked: [403, { detail: "Your role cannot log LINE_STITCHING. " +
    "Permitted: stitching_manager." }],
  ambiguousLot: [409, { detail: "3 LEATHER lots match that spec – say which one. " +
    "Add a thickness, or send the lot id directly. " +
    "Candidates: SHEEP GLASS · BLACK · 0.7 (2365.0 dcm " +
    "available, lot a1338000-...); ..." }],
  noLot: [404, { detail: "No LEATHER lot in stock for SHEEP GLASS · BLACK. " +
    "Add the delivery with POST /materials before " +
    "cutting from it." }],
  dcmDisagree: [422, { detail: "The pieces in this batch have different leather " +
    "consumption in their style specs (34.5, 36.0), so " +
    "there is no one quantity to record." }],
  notPresent: [422, { detail: "Ramesh Kumar is not checked in today." }],
  // store
  wrongDrawer: [409, { detail: "PC-004821 belongs to drawer DRW-0042, not DRW-0117." }],
  noEmployee: [422, { detail: [{ loc: ["body"], msg: "Scan the employee barcode " +
    "first – provide employee_barcode or employee_id.",
    type: "value_error" }]}],

  // release
  liningMissing: [422, { detail: [{ loc: ["body"], msg: "2 style(s) have no lining " +
    "answer: .... Set needs_lining true or false on every " +
    "style – it cannot be changed after release.",
    type: "value_error" }]}],
  styleReleased: [409, { detail: "CLERMONT is RELEASED – its pieces carry printed " +
    "barcodes and cannot be edited." }],

  // wages
  runOverlap: [409, { detail: "A run already covers 2026-09-01 → 2026-09-15 for " +
    "BOG-CLERMONT (run 70001111-...). Delete it if it is " +
    "wreckage from a failed compute, or narrow this window." }],
  pieceNoScope: [422, { detail: [{ loc: ["body"], msg: "A piece-rate run must name " +
    "the work it pays for: send style_code or " +
    "order_number.", type: "value_error" }]}],
  closedRun: [409, { detail: "This run is CLOSED. Reopen it (with a reason) " +
    "before recomputing." }],

  // clients
  clientHasOrders: [409, { detail: "BOGGI MILANO has 3 orders. Deactivate instead: " +
    "PATCH /clients/{id} with {\"is_active\": false}." }],

  // removed
  goneRoute: [410, { detail: "POST /production/cutting is removed. Pieces mint at " +
    "breakdown upload; log cutting via POST /production/log " +
    "with screen_context=LEATHER_CUT." }],

  // generic
  serverError: [500, { detail: "Internal server error",
    request_id: "3f9a2b71-8c4d-4e5f-a012-b3c4d5e6f708" }],
};
```


41. Frontend call sequences

41.1 Order to released garments

```
POST    /clients                                {name, country, order_number}
POST    /imports/preview                        (multipart: order_number + file) → review warnings
POST    /imports/commit                        (same) → release_required: true
GET      /imports/breakdown/{order_number} → the editable table
PATCH  /imports/breakdown/skus/{id}          {qty_ordered, style: {...}} (repeat as needed)
PUT      /styles/{style_id}/material-spec {lines: [...]}
GET      /styles/{style_id}/material-spec/requirement → short_by? order it
POST    /styles/{style_id}/material-spec/confirm {no_accessories: false}
GET      /drawers/pool → is the pool big enough?
POST    /imports/breakdown/{n}/release {styles: [{style_id, needs_lining}], grow_drawer_pool}
POST    /barcode/print {order_id} → print the labels
```

41.2 A cutting shift

```
POST    /attendance/scan-check-in {employee_barcode, direction: "in"} xN at the gate
GET      /materials/lots?category=LEATHER&sku_id=...&required=1035
→ options fill the dropdowns; last_used_for_sku pre-selects
GET      /barcode/resolve?code=EMP-000123 → the worker card
GET      /production/piece-state?code=PC-004821&employee_barcode=EMP-000123
→ suggested_dcm_per_piece prefills the field
POST     /production/log {actor, targets: {piece_barcodes: [...]},
                        work_date, consumption: {article, colour,
                        thickness, dcm}}
→ read consumption_recorded + stock_warning
```

41.3 The store

```
GET      /barcode/resolve?code=EMP-000123 → next_expected_scan: "DRAWER"
GET      /barcode/resolve?code=DRW-0042 → next_expected_scan: "PIECE"
POST     /drawers/store-scan {employee_barcode, drawer_barcode,
                             piece_barcode}
→ part_inferred, auto_received, kit
GET      /drawers?sendable=true&sort=recent → the send queue
POST     /drawers/send {drawer_ids: [...]}
→ count_sent, not_ready[], pieces_released[]
```

41.4 The pipeline

```
GET      /production/piece-state?code=PC-004821&employee_barcode=EMP-000124
├─ ready_to_log: true → POST /production/log
├─ ready_to_log: false → render blockers[]
└─ ready_to_log: null → "scan the worker's card"
POST     /production/log {actor, targets, work_date}
→ blocked[], skill_warnings[], sku_progress.closed
```

41.5 Payroll

```
GET    /wages/orders                    → the n-of-m badge
GET    /wages/styles?order_number=BOG-SS27-001
GET    /wages/rate-sheet?style_code=BOG-CLERMONT    → find rate: null rows
POST   /wages/rates/bulk                {style_code, effective_from, lines}
POST   /wages/runs {run_kind:"piece", style_code, period_start, period_end, freeze:false}
      → CHECK unrated_operations + gap_days
GET    /wages/runs/{id}/breakdown        → the three folds
GET    /wages/runs/{id}/pieces           → per-piece detail
POST   /wages/runs/{id}/close
      later, to correct:
POST   /wages/runs/{id}/reopen           {reason: "..."}
POST   /wages/runs/{id}/recompute
POST   /wages/runs/{id}/close
```

41.6 Tracing one garment

```
GET /barcode/resolve?code=PC-004821    what is it, where next
GET /analytics/pieces/PC-004821/story    the full life story
GET /dashboard/pieces/PC-004821         stage history + STORE overlay
GET /production/piece-state?code=PC-004821  what happens next, which cards to lock
```

42. Quick reference card

The five gates between an order and a garment

Gate	Blocks	Unblock with
Recipe not confirmed	Release	POST /styles/{id}/material-spec/confirm
Lining not declared	Release	needs_lining on every style in the release body
No drawer	Storage, then the merge gate	POST /drawers/pool or /allocate-waiting
Drawer not sent	LINE_STITCHING	POST /drawers/send
Not present today	Any production log	POST /attendance/scan-check-in

The four production gates

#	Gate	Scope	Failure
1	ROLE	Whole request	403
2	SKILL	Per piece	Warning — still logged
3	SEQUENCE	Per piece	sequence_blocked
4	MERGE (<i>LINE_STITCHING</i> only)	Per piece	merge_blocked

Never send

```
× a production stage      - derived from the screen or the history
× screen_context          - derived from the role (DM/MD/HR may override)
× a computed total        - available, amount, any count
× a device timestamp for a punch - the server stamps it
```

The three "the system is asking you" moments

Signal	Where	Meaning
409 with named lot candidates	Cut log	"Several lots match. Which one?"
kit.unresolved[] with AMBIGUOUS	Store scan	"Several lots carry this article. Pick one."
needs_lining: null	Release	"Nobody has answered this. I will not guess."

Each is a deliberate refusal to guess. Give each a first-class UI affordance — they are where the system's accuracy comes from.

The numbers that flow through everything

RELEASE	qty_ordered → N Piece rows → N barcodes → N drawer merges
CUT	dcm_per_piece × piece_count → lot.on_hand decremented ONCE
RECIPE	qty_per_piece × qty_ordered → required → short_by
WAGES	ProductionEvent rows × rate-in-force-that-day → wage_line.amount

Two things that are never true

- **A retired barcode 404s.** It 410s. They mean different things.
- **A partial batch is a failure.** 29 logged and 1 blocked is a good outcome — read the buckets.